UNIVERSITY OF MILANO-BICOCCA

Department of Computer Science, Systems and Communication

Master's Degree Programme in **Computer Science**

Innovative Management of Service Requests: From a Distributed Monolith to a Large-Scale Microservices Architecture, with RAG-Enhanced Reccomendation System

Advisor: Prof. Leonardo Mariani

Master's Thesis by:
Nicolas Guarini
Student ID: 918670

Academic Year 2024–2025

"What I cannot create, I do not understand"

- Richard Feynman

Contents

Introduction

This thesis describes the redesign and restructuring of a legacy system for managing Service Requests for an internship at Elmec Informatica, evolving it from a legacy distributed monolith with lots structural limitations (SRP) into a modern, scalable microservices architecture (ServiceRequestManager, SRM).

The legacy system is characterized by a fragmented architecture that compromises scalability, maintainability, and reliability. Key challenges identified includes a monolithic frontend component exceeding 11,000 lines of code where business logic is hard-coded into the presentation layer; a heavy dependency on the Agent installed on customer Domain Controllers, leading to security risks and operational bottlenecks; the use of unversioned logic stored within Stored Procedures and SSIS jobs, making collaboration and error tracking difficult; and significant data integrity issues within the legacy MSSQL database.

To overcome these obstacles, the new SRM architecture leverages a modern technological stack. The backend is implemented using Django and Django REST Framework, utilizing Celery and Redis for asynchronous task management. The system is containerized with Docker and orchestrated via Kubernetes, ensuring high availability and scalability. Furthermore, the dependency on legacy agents is eliminated by integrating an Ansible-based provisioning tool that allows secure, agentless remote execution.

A major innovation of this work is the implementation of an intelligent recommendation system. As the service catalog expanded to over 300 distinct actions, traditional hierarchical navigation became a source of cognitive load for users. This thesis introduces a Retrieval Augmented Generation (RAG) solution using an on-premise Large Language Model, allowing users to discover services using natural language with transparent reasoning to ensure trust and explainability.

The thesis is organized as follows: Chapter 1 analyzes the existing system and its flaws; Chapter 2 outlines the architectural redesign and requirements; Chapter 3 focuses on the practical implementation including CI/CD pipelines and the three-level configuration hierarchy; Chapter 4 describes the phased migration strategy maintaining business continuity; Chapter 5 details the RAG-based recommendation system.

Chapter 1

Overview of the existing System

The internship project was not focused on creating an entirely new system from scratch, but rather on a deep redesign and restructuring of an existing system developed many years ago using legacy technologies and affected by several issues and critical limitations.

Before discussing the architectural, technological, and implementation choices that were made, it is necessary to carry out a thorough analysis of the current system, in order to understand its behavior, operational flows, technologies used, the main issues to be addressed, how to resolve them, how to prevent them from reoccurring in the future, and how to ensure that the new system is scalable and maintainable.

Before proceeding with the analysis of the legacy system, it is essential to clarify the distinction between two fundamental concepts that will recur throughout this thesis: *Actions* and *Service Requests*.

An **Action** represents the formal definition and configuration of an IT service that can be executed. It specifies all the metadata necessary for execution, including the description, required input fields with their metadata, execution scripts, approval requirements, and so on. Actions constitute the catalog: the repository of all operations that customers can request through the system.

A **Service Request** (SR) represents the concrete instantiation and execution of a specific Action. When a user submits a Service Request, they are creating an executable instance of an Action by providing values for its required fields. The Service Request follows a complete lifecycle from creation through validation, potential approval, execution on target infrastructure, and final completion or error handling. While an Action defines "what can be done," a Service Request represents "what is actually being done" at a specific moment for a specific customer or resource.

1.1 Service Requests Workflow

The lifecycle of a Service Request within the current system is orchestrated through a series of defined states and automated processes, involving interac-

tions between various components and human operators. The flow is designed to manage SRs from initiation to completion, handling approvals, execution, and potential errors [40].

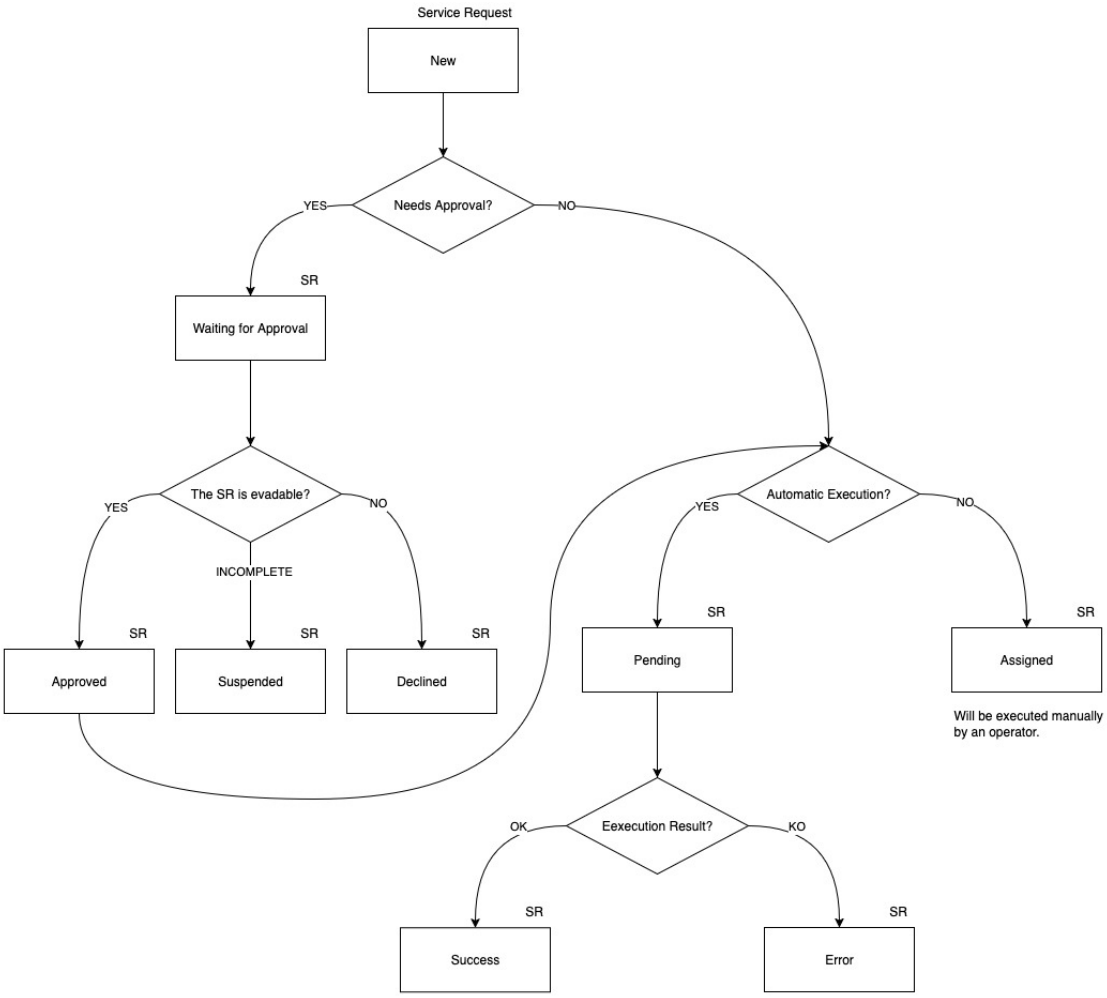

Figure 1.1: High-Level Service Request Lifecycle

In Figure 1.1 [40], a simple and high-level flowchart is shown that illustrates the main flows and scenarios.

1.1.1 SR Initiation and Initial State Management

A Service Request can be initiated primary through two channels [40]:

- **Via MyElmec Portal**: Clients can submit an SR by completing a dedicated form from the MyElmec portal. Upon saving the form, a ticket is created in the HelpDeskAdvanced (HDA) system with a `QUALIFIED` status, and a corresponding Service Request is generated in SRP with a `NEW` status. These two entities are then linked via the HDA Ticket ID;
- **Via HDA Ticket by Elmec Operator**: Operators have the ability to associate a catalog SR directly with a ticket they are managing within the

HDA interface. When the ticket is saved in a `DELEGATED` status, the SR is created in SRP with a `NEW` status, linked to the ticket, and immediately triggers the SR workflow within SRP.l

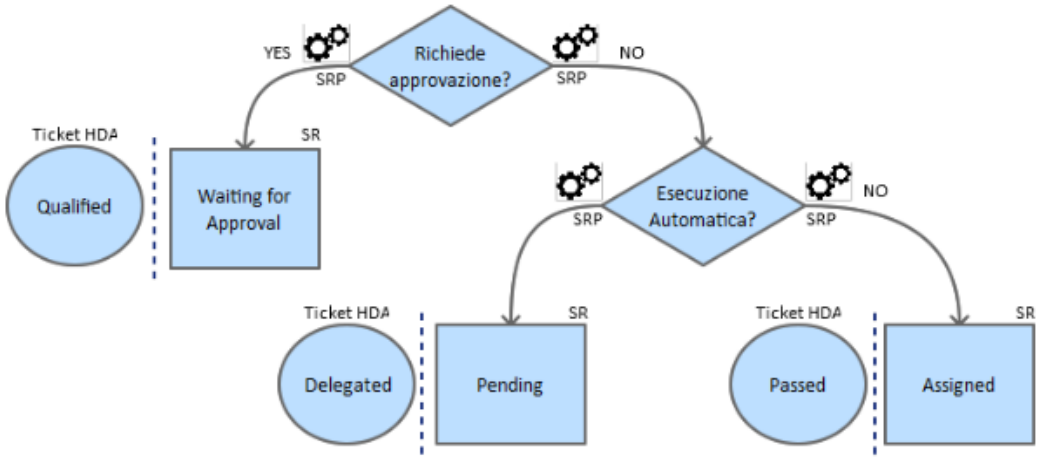

Figure 1.2: AS-IS New Service Request Workflow

As shown in Figure 1.2, following the Service Request creation, the system evaluates whether the Service Request requires approval by the operators. The logic for setting the initial SR and HDA ticket states is as follows [40]:

- **If approval is required**: the Service Request is set to `WAITING FOR APPROVAL` status, and the corresponding HDA ticket is set to `QUALIFIED`. A communication is added to the ticket, indicating that the SR needs validation from an operator before execution;
- **If no approval is required and it's linked to an automatism**: the Service Request enters a `PENDING` state, and the HDA ticket is set to `DELEGATED`;
- **If no approval is required and it's not linked to an automatism**: the Service Request is `ASSIGNED`, and the HDA ticket is set to `PASSED`. A communication is added to the ticket, noting that the SR requires manual processing by an operator.

1.1.2 Approval and Execution Workflows

As shown in Figure 1.3, for SRs requiring approval (`WAITING FOR APPROVAL` state with an HDA `QUALIFIED` ticket), an operator's intervention is necessary to validate the request. The operator has three possible actions [40]:

- **Cancel the SR**: the HDA ticket is set to `CLOSED ABORTED`, the SR status becomes `DECLINED`, and a notification email is sent to the requesting client;
- **Request client modification**: the HDA ticket is set to `WAITING USER`,

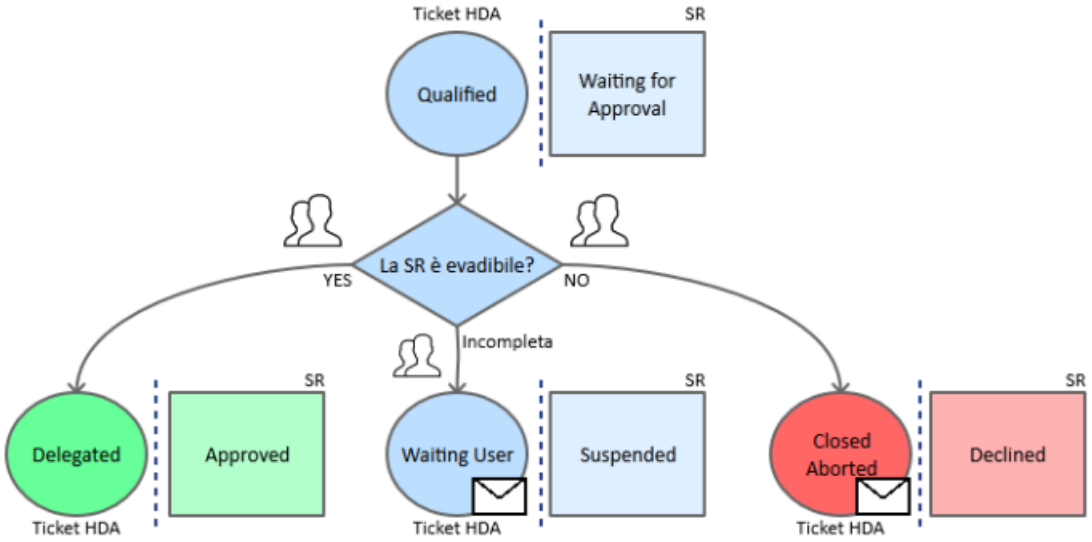

Figure 1.3: AS-IS Approval Service Request Workflow

the SR status becomes `SUSPENDED`, and a notification email is sent to the client. The workflow pauses until the client modifies the SR via the MyElmec platform;

- **Approve the SR**: the HDA ticket is set to `DELEGATED`, and the SR status becomes `APPROVED`.

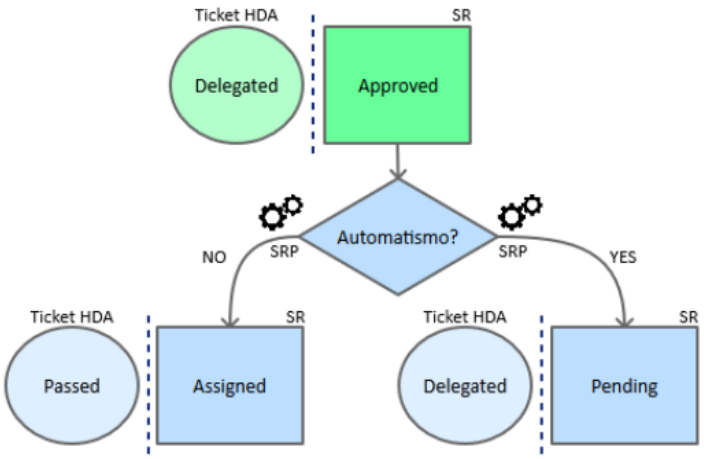

Figure 1.4: AS-IS Approved Service Request Workflow

As shown in Figure 1.4, once a SR is approved, SRP manages its subsequent flow based on whether it is linked to an automatism [40]:

- **If the SR is linked to an automatism**: the SR transitions to a `PENDING` state, while the HDA ticket remains `DELEGATED`. This implies it's awaiting automated execution;

- **If the SR is not linked to an automatism**: the SR is immediately set

to ASSIGNED, and the HDA ticket is moved to PASSED. A note is added to the ticket indicating that the SR requires manual processing.

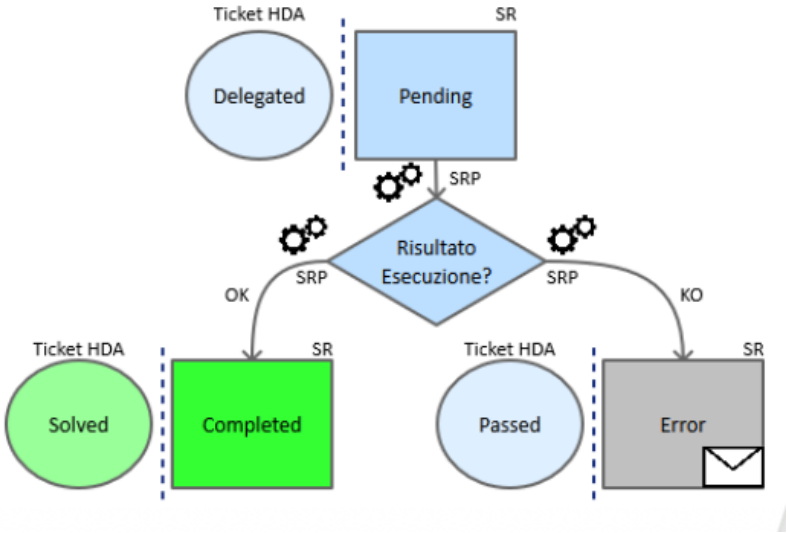

Figure 1.5: AS-IS Pending Service Request Workflow

As shown in Figure 1.5, an SR in PENDING state is awaiting the execution of its associated automatism. The outcome of this automated execution determines the next state [48]:

- **Successful execution**: the SR is set to COMPLETED, and the corresponding HDA ticket is set to SOLVED (and then to CLOSED);
- **Unsuccessful execution**: the SR transitions to an ERROR state, and the HDA ticket is set to PASSED. Communication regarding the error is added to the ticket for visibility within HDA. The SR will be then handled manually by an operator.

The execution of Service Requests themselves can involve the SRP Agent on the client's Domain Controller for operations requiring client-side interaction (e.g., Network, Active Directory or Exchange environment changes, for example). The SRP Agent (installed on every enabled customers' Domain Controller) periodically checks ELMEC-SRP01 (a on-premise server hosting the MSSQL DB and the core business logic) for pending SRs. If found, it retrieves the SR details, donwloads the relevant PowerShell script, and executes it. The agent then informs ELMEC-SRP01 that the SR is running [48]. Monitoring is performed by a scheduled task on the client side, which checks the PowerShell script's log file every minute. A "KEY" in the log indicates completion, prompting the SRP Agent to inform ELMEC-SRP01 to set the SR to COMPLETED, triggering an HDA status update. An "ERROR" key indicates failure, leading to the log being sent to ELMEC-SRP01 and an error signal. A timeout mechanism also signals an error if completion is not detected within a specified duration [48].

A constraint that must be respected is that the high-level flow (state transitions and mapping between SR states and HDA ticket states) must not be modified, as it represents an established and approved process that also encompasses organizational and accountability aspects among various Elmec departments [41]: those who configure and manage the catalog and related Service Requests (Managed Services, MxS, team), those who develop and maintain the product (Innovation team), and those responsible for the ticket management system (Customer Process Integration, CPI, team). The technological and implementation aspects, on the other hand, can and should be redesigned and improved [41], as will be explored in the following chapters.

1.2 Current System Architecture

The *SRP* system architecture is fundamentally distributed into two primary segments:

- Company Network;
- Client Network.

These segments interact and communicate exclusively through a *DMI Console* [42], which acts as a unidirectional bridge, transferring requests from the client network to the internal company network [42] [17].

The overall architecture is depicted in the diagram in figure 1.6 [42].

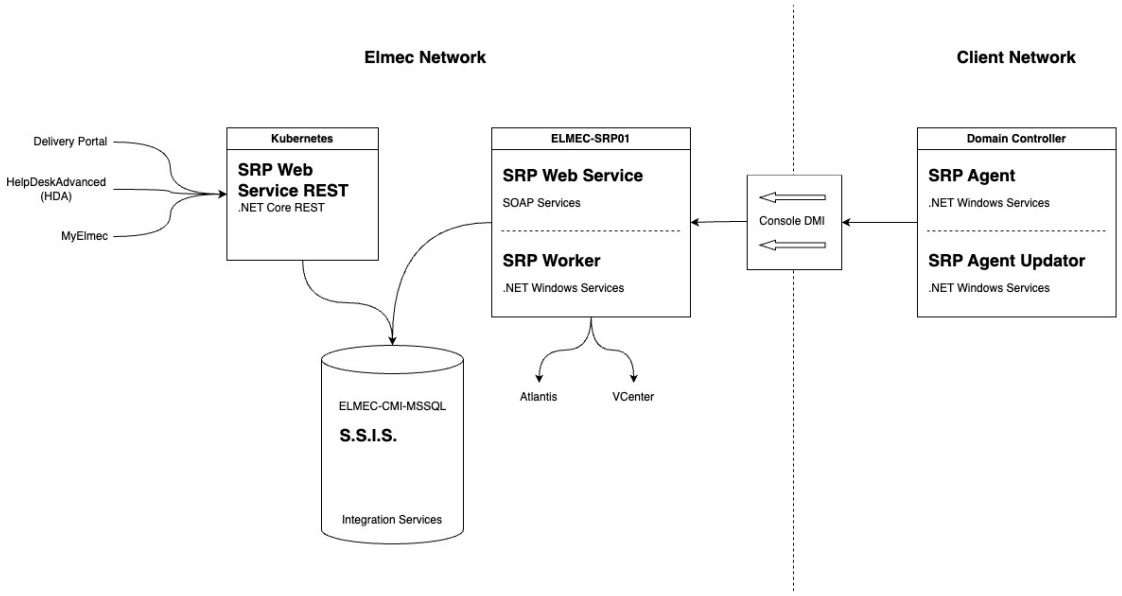

Figure 1.6: AS-IS System Architecture

The system's operational flow involves several key components across both networks, orchestrating the processing of service requests.

1.2.1 Main Functionalities and Components

The current *SRP* system relies on a suite of interconnected components, each serving a specific role in the lifecycle of a service request.

Elmec's Network Components:

- **SRP Web Sevice REST** [47]: Deployed on a Kubernetes cluster, exposes .NET Core RESTful services [42]. It serves as the primary interface for external systems such as:
  - **Delivery Portal**: An internal asset management system used to manage Service Request catalog configuration and Elmec's internal and client servers;
  - **HDA (HelpDeskAdvanced)**: An internal ticket management system. Each service request is intrinsically linked to an HDA ticket via an ID, and HDA is responsible for initiating and associating service requests with existing tickets.
  - **MyElmec**: A customer-facing portal enabling customers to create, manage, and monitor the various services provided by Elmec (e.g., servers, management software, cloud solutions).

  The *SRP Web Service REST* communicates with the *ELMEC-CMI-MSSQL* server.
- **ELMEC-CMI-MSSQL**: This server hosts the central SQL Server Database for the SRP system. It is the persistent storage layer for all service request data and related information [42].
- **S.S.I.S. (SQL Server Integration Services)**: Running on the `ELMEC-CMI-MSSQL` server, SSIS packages are crucial for data transformation and import/export operations. They handle the transfer of data, such as master data (e.g., Active Directory users, Organizational Units, AD Groups) sent by SRP Agents, into the database. Each SSIS package is designed to transform the received information for database insertion or vice versa [42].
- **ELMEC-SRP01**: This Windows Server functions as the core engine for service requests within the Elmec network. It houses all the PowerShell scripts associated with individual service requests, which are maintained by the MxS (Managed Services) team [42]. Key components residing on `ELMEC-SRP01` include:
  - **SRP WebServiceSoap**: This component provides SOAP services primarily for communication with the SRP Agent located on the client side. It interacts with the database to retrieve necessary information before exposing it to the agents for Service Request execution [46].

- **SRP InternalWorker**: A Windows service that operates similarly to the SRP Agent but executes entirely within the Elmec network. This is utilized for service requests that do not require interaction with the client's network or Active Directory (e.g., managed VM-related service requests). The InternalWorker queries the database every 10 seconds for "internal worker" type jobs (like removing a server from a patching session, or resetting a managed Virtual Machine), connects to the SOAP service, retrieves the Service Request ID and its fields, and initiates job execution. It also monitors logs every 30 seconds to determine the job's status [45].

The only component in the client's network is called **Domain Controller**, which represents the client's Active Directory domain controller, where the communication tools with Elmec are installed. It hosts:

- **SRP Agent (.NET Windows Service)**: Typically, one agent is configured per client domain, specifically for Active Directory (AD) and Exchange-related service requests. This agent performs two main functions [48]:
  - Every 8 hours, it reads master data (AD users, OUs, groups) from the domain controller and sends it to `ELMEC-SRP01` for storage via SSIS.
  - Every 5 minutes, it queries `ELMEC-SRP01` for pending Service Requests. Upon finding any, it requests detailed information, loads the corresponding PowerShell script, and executes it. After initiating the script, the agent informs `ELMEC-SRP01` that the SR is "running" and proceeds to the next SR.
- **SRP Agent Updator (.NET Windows Service)**: This service continuously monitors the SRP Agent folder for a file with a `.new` extension. If detected, it stops the SRP Agent, updates it, and restarts it

1.2.2 Interactions and Communication between Customers and Company networks

The Communication between the company network and the customers' networks is a critical aspect of the SRP system's operations and is handled through the **DMI Console** [42].

The DMI console serves as a key infrastructural component, primarily acting as a communication bridge between the customer's network and Elmec's internal network. Specifically, for the SRP system, the DMI console is responsible for transferring service requests (SR) from the customer's network to Elmec's internal network, although it does not handle traffic in the opposite direction

[17].

Operation of the DMI Console

The connectivity of the DMI appliances to Elmec is established through multiple OpenVPN connections, initiated by the appliance towards the Brunello 4 and Mendrisio Data Centers (for Swiss customers). The DMI consoles are not directly exposed to the Internet and are accessible only by authorized personnel [17].

Following recent developments in the DMI, the new consoles use K3S for container management. K3S is a lightweight, certified Kubernetes distribution designed for edge, IoT, and CI/CD environments. It is characterized by being a single binary that reduces external dependencies and uses SQLite as the default database, greatly simplifying installation and management. It already includes essential components such as *containerd* [22].

All consoles are centrally managed via Rancher, and the deployment of the application stack is delegated to ArgoCD, which ensures that container versions are constantly updated. For security reasons, the console in the customer network by default exposes only SSH, while services such as SRP are selectively enabled and only when necessary. All application data present on the consoles is stored on a LUKS (Linux Unified Key Setup) encrypted volume[1], which can only be unlocked if the VPN tunnel is established. The operating system update is performed through a standard Elmec system, called PMD (Patching Management Dashboard) [17].

1.3 Problems and core issues of the current system

The analysis of the architecture and workflow of the current SRP system has revealed a number of critical issues that compromise its scalability, maintainability, and reliability. These problems are deeply rooted in the technological and design choices made years ago and are evident across several key areas of the system, from catalog management to script execution and the import of master data from Active Directory.

[1]A platform-independent hard disk encryption method that can be used to encrypt disks across a wide variety of tools. This not only ensures full compatibility and interoperability between different software but also guarantees that password management occurs in a secure and documented manner [13].

1.3.1 Architectural and Technological Challenges

The SRP system is characterized by a fragmented architectural structure and complex interdependencies between its main components, which leads to several critical issues.

Monolithic Frontend Component

The user interface, responsible for navigating the Service Request catalog and completing the associated forms, is an excessively large and complex Vue.js frontend component (nearly 11,000 lines). This architecture makes modifications, extensions, and client-specific customizations exceptionally challenging [32]. The logic for managing dynamic fields, their interdependencies, and the rules for compilation and validation are all handled at the frontend level through a long series of hard-coded conditional statements, such as the one shown in Listing 1.

```
<div v-if="field.desc == 'Domain'">
    ...
</div>
```

Listing 1: Example of field rendering logic in legacy monolithic frontend code

This, in addition to mixing business logic with presentation, represents a significant challenge for catalog configuration, as there is no centralized way to manage the fields and their relationships. Every change, therefore, requires a direct intervention on the code, making the system inflexible and difficult to maintain, especially in a context where the operator modifying the catalog is not a programmer and does not know how the frontend code is structured. For instance, if an operator wanted to add a new "Domain" field to a Service Request, this field would need to have the exact description "Domain" and type "domain"; otherwise, the Vue.js component would not render it correctly.

These dynamics make the system fragile and susceptible to frequent errors that waste time and resources for both clients and operators [32].

Overly Generic Backend REST Service

The only information about the fields returned by the backend REST service (`SRPWebSeviceREST`), besides the field name and its mandatoriness, is its *type*. It is therefore easy to imagine how this information, which should only be an indication of the component that needs to be rendered (e.g., textbox, select, radio button, etc.), can become an identifier for something much broader.

For example, a field of type `UPN` is a composite field derived from the value of another field, '`Logon Name`', concatenated with an '`@`' symbol, and suffixed with the value from a select field whose options are the client's domains, provided by an external REST service (e.g., "`n.guarini@elmec.it`", where

"n.guarini" is the value of the "Logon Name" field, and "elmec.it" is the value of the "Domain" select field).

Despite this complexity, the REST service returns only a JSON document like the one shown in Listing 2.

```
{
    "idField": 1603,
    "descField": "UPN",
    "type": "upn",
    "mandatory": true
}
```

Listing 2: Example of field data returned by legacy SRP System REST Service.

thus delegating to the frontend the responsibility of interpreting the type and rendering the field correctly.

Dependency on the SRP Agent

The import of data from clients' Active Directories and the actual execution of Service Requests are handled by an agent (`SRPWindowsService`) installed directly on the clients' Domain Controllers. This approach presents several challenges:

- **Security and Permissions**: The agent often requires credentials with elevated privileges (e.g., `Domain-Admin`), which clients are increasingly reluctant to grant.
- **Reliability and Control**: The configurations on the agent and the Domain Controller are complex and difficult to manage centrally, which can cause execution errors and data synchronization issues.
- **Scalability**: The need to install, configure, and maintain an agent for each client domain represents a significant operational effort. Alternatives such as more modern provisioning systems will be evaluated.
- **Obsolete Communication Patterns**: The unidirectional communication through the *DMI Console* creates a bottleneck and complicates the management of asynchronous operations. Communication between components in the client's network and the company network is based on polling (e.g., the SRP Agent querying the server every X second/minutes), an approach that introduces latency and inefficiencies.

1.3.2 Code and Data Management Issues

Code and data persistence management is another critical aspect of the system, characterized by a lack of versioning and modern debugging tools.

Unversioned and Undebuggable Stored Procedures

The intensive use of Stored Procedures[2] in the MSSQL database represents one of the major weaknesses. These procedures contain fundamental business logic and are:

- **Unversioned**: There is no centralized version control system for Stored Procedures. Changes are made directly on the database, making it impossible to track changes, revert to previous versions, or test modifications in separate environments.
- **Difficult to Debug**: Debugging Stored Procedures is a complex and cumbersome process, often requiring direct access and specific SQL server tools, with the risk of affecting the production environment.

Inaccessible and Unversioned SSIS Jobs

SSIS packages[3] are essential for data import and export operations (e.g., AD master data). However, these jobs reside physically on the `ELMEC-CMI-MSSQL` server, and the only way to access and modify them is via a remote desktop. This approach prevents versioning and collaboration, limits accessibility by making maintenance and debugging an onerous and risky activity, and creates a *single-point-of-failure* and a strong coupling between the import/export business logic and the infrastructure.

1.3.3 Database Schema and Data Integrity Issues

The analysis of the database schema, has revealed a number of significant issues that affect the quality and integrity of the data managed by the SRP system.

Lack of Data Integrity and Consistency

One of the most significant problems concerns the lack of explicit referential integrity constraints. Although the relationships between tables have been inferred manually, they are not defined at the database level through the use of foreign keys. This absence prevents the database from guaranteeing data

[2]Stored procedures are SQL code routines, or sets of instructions, that are pre-compiled and stored within a relational database. They act as logical units of execution, encapsulating business logic directly at the database level [50].

[3]SQL Server Integration Services (SSIS) is a platform for building data integration and workflow solutions. It is a key component of Microsoft SQL Server, primarily used for Extract, Transform, Load (ETL) operations, enabling the extraction of data from various sources, its transformation, and its loading into different destinations [44].

integrity [7]: for example, a row in a child table could refer to a non-existent row in the parent table [61]. Such a scenario, known as a "dangling reference[4]", can generate anomalies and inconsistencies in the system, making the data unreliable and debugging extremely complex [7].

Data Redundancy

Another issue is the high data redundancy and the large number of duplicate attributes across different tables [48]. This design, which deviates from the principles of normalization [2], leads to storing the same information in multiple locations, which not only increases the data volume but also introduces a high risk of inconsistencies [2].

Unclear and Inconsistent Naming Conventions

The naming convention for attributes and tables is inconsistent and unclear. The schema presents a mixture of conventions (e.g., `IDGroup` vs. `GroupID`, English names mixed with specific terms) and typos (e.g., `TBParamenters`), making it difficult to immediately understand the function of each field. In many cases, the naming is not explicit enough to describe the role of an attribute, requiring an in-depth knowledge of the system to correctly interpret the relationships and business logic. This lack of a unique and standardized vocabulary hinders collaboration among developers, complicates code maintenance, and increases the likelihood of errors.

1.3.4 Maintenance and Operational Challenges

According to information discussed in an internal meeting [41], the daily management of the system is made complex by several inefficiencies and obsolete design choices.

The current system design does not allow for the effective reuse of Service Requests and catalog configurations. Consequently, every modification or addition requires the creation of a new Service Request, even for small variations, leading to a innefficient and difficult-to-manage catalog, consisting of numerous duplicate SRs and fields. This, while functional, is not a scalable approach and leads to significant inefficiencies in catalog management itself. For example, if an Action needs to be modified for all clients who use it, the operator will have to modify not only the action in question but also all the

[4]The "dangling pointer" is a problem typically addressed in the context of low-level programming, where a pointer refers to a memory area that is no longer valid because it has been freed or because the pointer is used outside the context of the variable's existence to which it refers [19], but the same dynamic can also be applied in the context of databases, where the "pointers" are represented by foreign keys.

related duplicates for each client, with the risk of forgetting a modification or making mistakes. This approach makes the catalog extremely difficult to navigate and maintain.

The new architecture will need to address these challenges by introducing a more robust, versioned system based on modern software design principles and patterns.

Chapter 2

Architectural Redesign

Given the numerous critical issues that emerged from the analysis of the current system, it is clear that a deep redesign is necessary. This redesign must be able to perform the system's current functions more efficiently and in an optimized manner, while also satisfying a series of requirements that did not exist when the system was developed several years ago, and which the current system cannot manage because it was not designed to do so.

2.1 New System Requirements

The system can be divided into three main components:

- **Catalog** (Structure and Management);
- **Service Request Execution**;
- **Master Data Import / Export**.

The requirements for each of these areas will therefore be analyzed.

2.1.1 Catalog Structure and Management

This is the most substantial component of the three, dealing with everything concerning the definition, maintenance, and configuration of the actions executable by each customer. This therefore also includes the management of form fields, customer-specific customizations, scripts, their parameters, the target machine address retrieval logic, and so on.

Like mentioned in the first chapter's introduction, an '*Action*' is understood as the formal definition of an operation that a client can execute on its infrastructure, while a '*Service Request*' (SR), on the other hand, is understood as the effective execution of an Action. If a parallel with OOP is to be drawn, the Action can be seen as a class, and the Service Request as an object, an instance of the aforementioned class.

Actions

The Action entity can be seen as the core of the catalog component, and it is composed of the following main elements:

- General metadata, such as the name, an extended description, tags, a type, a group, a category, and other information related to administrative and billing aspects;
- The fields of the corresponding form that must be filled by the client for its execution, including the relative logic tied to types, data validation, dependencies between various fields (e.g., fields that, once completed, "enable" other fields), autofill (e.g., fields whose values are autofilled with values from other fields), sensitive values, and so on;
- The script that will be executed in the customer's infrastructure to perform the effective required operation;
- Script parameters, which may be field values, but also additional parameters necessary for execution (e.g., AWS credentials, 365 credentials, etc. . .);
- The target machine on which the script must be executed.

Some of these dynamics are already implemented in the current system (with all the already discussed issues), while others are not. The crucial point, however, is that all these configuration aspects must be customizable for specific customers and for specific actions [41].

Given that the users of this system are very large clients (Fendi, Valentino, Lavazza, to name a few), it is easy to imagine that each of them has highly specific needs, which make direct reuse of Actions across multiple clients impossible. At the same time, however, the creation of infinite duplicated Actions that share almost everything, with only small differences (as happens in the current system [41]), must be avoided.

It is therefore necessary to implement a system that provides for the possibility of defining a sort of hierarchical structure, where a "parent" action is configured, which can be "inherited" by several child actions that will specify only the modifications, all without affecting the "default" action.

Fields

Together with actions, fields are central components on which a large part of the system is based. As already mentioned, fields are entities that can be extremely diverse, such as text fields, selects, multiple-choice selects, radio buttons, etc. In the current system, there are 86 different field types. This number is so high because, due to how the current system is built, the "type"

information is the only way the backend communicates to the frontend what type of field is to be rendered [47]. In fact, there are many different types that share the same logic, perhaps simply using two different services to retrieve the data. The effective field typologies are therefore much fewer, and it is crucial that they are identified and generalized as much as possible.

The general idea is that the logic for fields must be moved to the backend, which must return all the necessary information for the frontend to render the form: from field types, to validation regexes, to error messages, to any external services to be contacted to retrieve the options list, or to validate the data, and so on [41].

All this must obviously be compatible with the customization requirements specified in the previous section.

Regarding customizations, a typical example scenario could be the following: a customer uses an action, but for some reason requires two additional fields, which will then be needed by the script that will in turn require two additional parameters. Furthermore, for some fields, the client in question uses different validation rules and specific autofill templates. These customizations must be possible without modifying the default action and fields.

Regarding "particular" field types, the system must provide for [41]:

- Select fields whose options are customer-specific (e.g., "Domain" field, which is a select with all the customer's domains);
- Select fields whose options are global for the entire system (e.g., "Country" field, which provides a list of states that are the same for all customers);
- Select fields whose options are customer-specific and to obtain them an external service must be contacted, with the possible option to search (e.g., "Server" field, which is a select with all the customer's servers, returned by the Atlantis CMDB service);
- Autofilled text fields, meaning their values are obtained through a templating system (e.g., Jinja) that uses the values of other fields (e.g., "Logon Name" field, populated by concatenating the user's first and last name separated by, for example, a dot);
- Text fields whose validation occurs through regex (e.g., "Date" field, which must comply with the standard format);
- Text fields whose validation occurs through an external service (e.g., "Logon Name" field, which requires contacting the relevant Active Directory service for validation that a user with the same logon name does not already exist);

- Fields with sensitive values, whose values will therefore need to be saved encrypted, using some symmetric encryption algorithm (e.g., "Password" field);
- "Mixed" fields, composed of multiple "sub-fields" with different characteristics (e.g., "UPN" field, which is composed of the first letter of the first name, the last name, an at sign (`@`), and the value of a select whose options are obtained from an external API call);

It is important that the system is designed in a scalable way, so that if another field typology were to be added in the future, it can be implemented with the minimum possible effort.

Configuration and Execution Consistency

A system for monitoring the consistency of the catalog configurations [41] will be necessary, in order to prevent the creation of invalid configurations, which would then lead to errors during SR execution.

For example, if a `select` type field is specified for an action, whose options are obtained from customer-specific parameters, but those parameters have not been defined for that customer, the system must report this inconsistency through some alerting system (e.g., email, ticket, monitoring dashboard). Alternatively, if an action is configured to be executed on a certain type of machine but the field necessary to select it is not among the action's fields, the system must report this inconsistency.

Similarly, a system for monitoring the consistency of SR executions must be implemented, in order to be able to report any errors that may have occurred during execution, such as an SR stuck in an "intermediate" state (e.g., `NEW`) for too long, or an SR that started execution (i.e., in `PENDING`) but never reached completion (i.e., in `COMPLETED` or `ERROR`) within a certain time limit (e.g., 2 hours).

Additional script parameters

Another topic that must be managed is that of script parameters. In the old system, there was granular control over every single parameter, which was linked to an action's script and possibly to a field in the corresponding Service Request. In practice, all the action's fields with their respective values were passed as parameters to each script, plus some additional parameters not linked to any field (e.g., Exchange Server, AD Sync Server, various credentials). This dynamic of additional parameters occurred through the definition of parameters with the foreign key on the field set to `NULL`, which were then intercepted by the stored procedure relating to the generation of the parameter string which, with hard-coded `if` statements (e.g., `IF @ParamName = 'ServerExc' BEGIN ...`), were valued accordingly, retrieving the information with custom

and non-standardized logic.

In the new system, it will be necessary to standardize these dynamics, finding a way to define the additional parameters that will be needed, beyond those of the fields, during action configuration, and to generalize the implementation logic as much as possible, so that if new "groups" of additional parameters (e.g., AWS credentials, O365 credentials, etc.) were to be added, it can be done in the most efficient way possible.

Target Machine Configuration

In the old system, Service Requests were executed exclusively on the customers' Domain Controllers. Specifically, each DC ran an agent, which periodically checked via *polling*[1] if there were any Service Requests in the `PENDING` state with a matching `domain` field.

This part of the architecture will also need to be redesigned, as it is inefficient and highly limiting by design [55]. Therefore, in addition to redesigning these dynamics, it will also be necessary to provide the option to choose, during the configuration of an Action, to execute the related SRs on other types of machines, such as internal Elmec workers, or other types of machines on the customer's network other than the Domain Controller (e.g., Exchange Server). It must also be possible to choose to execute SRs through the old flow, in order to continue supporting both systems, at least initially [41].

2.1.2 Service Request Execution

In the current system, the execution of SRs occurs according to this sequence of main events:

1. The frontend sends the request to create a new SR to the REST service (`SRPWebServiceREST`), with all the necessary data (completed fields, customer, action, etc.) [47];

2. The SR and the relevant field values are created in the database, setting the status to `NEW` [47];

3. An SSIS job, every minute, checks if there are SRs in the `NEW` state, and if any are found, they are processed, moving them to subsequent states (e.g., `WAITING FOR APPROVAL`, `PENDING`, `ASSIGNED`, etc.) depending on the action and customer configuration [42];

[1] *Polling* is a communication technique where a system or device periodically and actively samples the status of an external resource or device to check for readiness or new data.

4. When the SR is in the `PENDING` state, it means it is ready to be executed by the agent on the customer's DC;

5. Every 10 seconds, the agent on the customer's DC checks if there are SRs in the `PENDING` state for its domain (through calls to the SOAP service (`SRPWebService`)) [48];

6. If any are found, for each one, the corresponding script is downloaded, executed, and the SR status is updated to `COMPLETED` or `ERROR` depending on the execution outcome (through the SOAP service) [48];

It has already been discussed in the previous chapter how this flow has numerous criticalities, which make it inefficient and not very scalable. It will therefore be necessary to completely redesign it, taking into consideration the possibility of executing SRs on machines other than DCs, and the possibility of executing them also on machines internal to Elmec, and even virtual machines [41].

Elmec already has an internal *provisioning* system, called *Provision Prime*, which is responsible for executing Ansible playbooks on machines (internal, external, and also virtual) [36], and which could be leveraged for this purpose. It will therefore be necessary to analyze this system, and understand if and how it can be integrated into the new SR execution flow, in order to avoid the polling of agents on the DCs, and to have a generally more reliable and efficient system [41].

A constraint that must be respected, however, is that the action scripts are all in PowerShell, and it is not possible to modify them. This is because they are very numerous, have been developed over the years, have been tested and validated for every single client, and are maintained by different Elmec departments. Therefore, even if the SR execution flow is redesigned, the scripts must be executed as they are. This obviously does not apply to new scripts that will be developed in the future, which can be designed leveraging the new technologies [41].

2.1.3 Master Data Import

Another important component of the system is the one responsible for importing customer master data, necessary for populating some action fields (e.g., UPN domains, user accounts, groups, Organizational Units (OU)[2], databases, etc.). The main difficulty lies in the fact that this information physically

[2]In Windows Server, an Organizational Unit (OU) is a container within Active Directory (AD) used to logically group objects like user accounts, computer accounts, and security groups, similar to a folder in a file system [1].

resides on the customers' machines (large enterprise clients, with thousands of user accounts and groups), and therefore their retrieval in real-time is not immediate.

In the current system, this import occurs in a manner very similar to the SR scenario, but in reverse. It happens through an agent running on the customers' DCs, which periodically (every hour at :20) (downloads and) executes a series of PowerShell scripts to retrieve the necessary information, and deposits them in a specific system folder (`D:\SRPortal\Anag\`) [48]. These files are then consumed by an SSIS job (one for each type of master data) that imports them into the database. The REST services then expose APIs to retrieve this information, which are used by the frontend to populate the action fields [47].

In the new system, it will be necessary to redesign this flow as well, in order to eliminate the dependence on agents on customer DCs. Even in this case, however, the PowerShell scripts cannot be modified [41], and must be executed as they are now. It will therefore be necessary to analyze how to integrate the execution of these scripts into the new flow, perhaps also leveraging Elmec's internal *provisioning* system (*Provision Prime* [36]) in this case, in order to obtain a more traceable, reliable, and efficient system [41].

2.2 New System Architecture

After gathering and defining the requirements for the new *ServiceRequestManager (SRM)* system through various meetings with stakeholders and a deep analysis of the current system (SRP), it has been possible to design a new architecture for the system that allows for efficient and scalable satisfaction of these requirements.

The proposed new architecture for the SRM system, shown in Figure 2.1, is based on a **microservices architecture**, which allows for greater modularity, scalability, and ease of maintenance compared to the distributed-monolithic architecture of the current SRP system [14].

2.2.1 Main Components

The SRM system architecture is composed of the following main components.

Frontend

Two separate web frontends will be developed for end-users and system administrators. These frontends will communicate with the REST services via HTTP/HTTPS APIs.

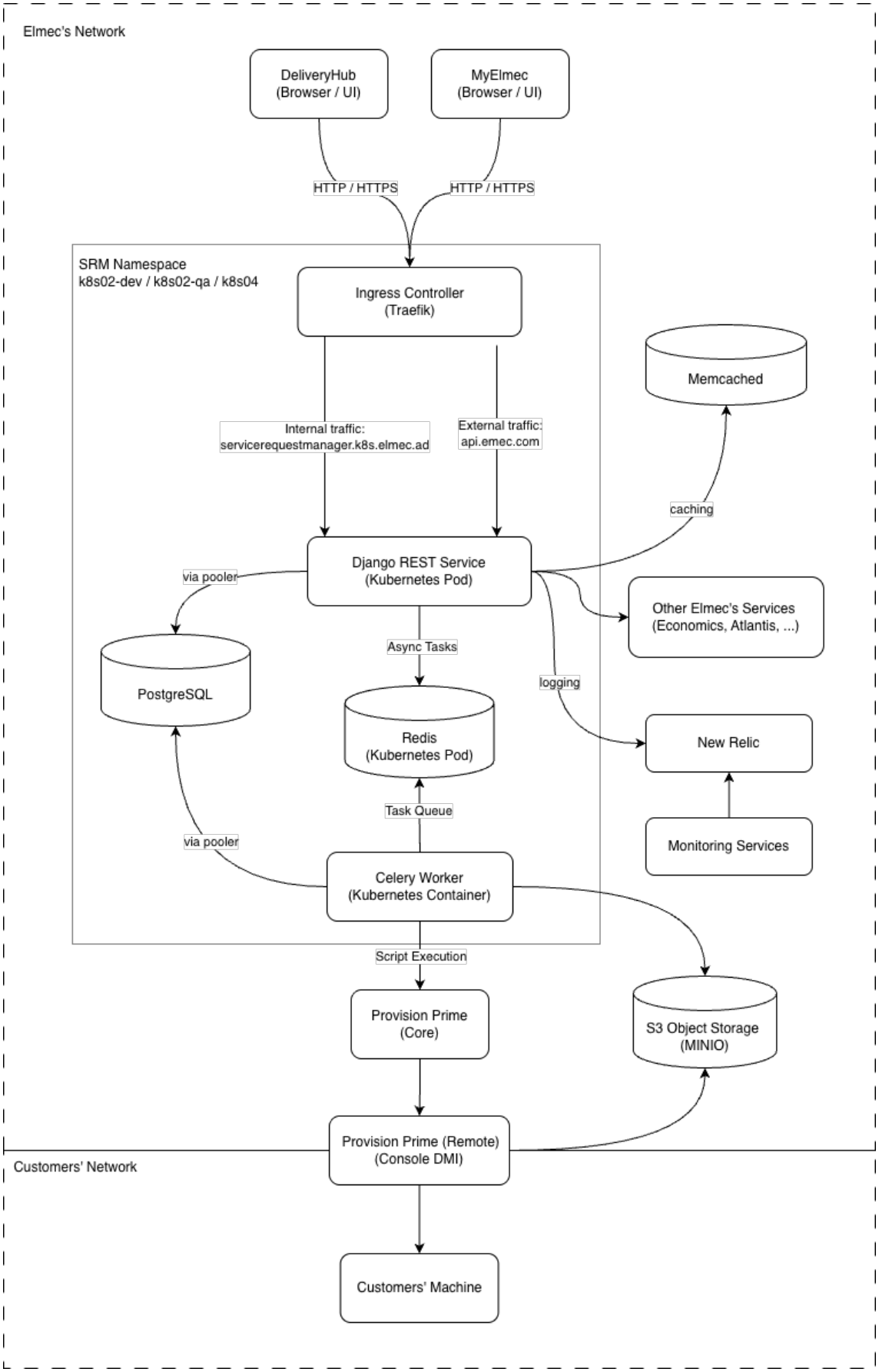

Figure 2.1: SRM System Architecture

Due to the business logic being moved to the REST services, the frontends will be lighter and easier to maintain, allowing for greater flexibility in implementing new functionalities and user interface improvements [32]. This also allows for the future integration of the (already existing) mobile application, which can use the same APIs.

REST Services

REST (Representational State Transfer) is an architectural style for designing web services that leverages HTTP protocols for communication between client and server [18]. The basic idea is that a system's resources are exposed through a uniform interface, typically HTTP, using [18]:

- URLs to identify resources (e.g., `/api/catalog/actions/123/`);
- HTTP verbs to operate on resources (`GET`, `POST`, `PUT`, `DELETE`, ...);
- Representations of resources in standard formats (JSON).

These services will be responsible for business logic, user request management, and interaction with databases and other system components.

Authentication and Authorization System

Authentication and authorization will be managed through the internally developed library, PyElmecAuth or GoElmecAuth (depending on the implementation language), which will interface with the corporate identity management system (ADFS) to ensure secure and controlled access to the REST services.

It will therefore be necessary to integrate a granular permission system into SRM to ensure that users can only access the resources and functionalities for which they are authorized, based on the roles defined by the corporate Identity Provider.

Caching System

A caching system will be required to improve system performance and reduce the load on the databases.

In particular, responses from calls to external services will need to be cached, such as third-party APIs needed to validate some fields, or calls to retrieve customer information, to reduce response times and improve the user experience. The time-to-live (TTL) of the caches must be configurable based on the specific needs of each data type.

Message Broker and Task Queue

To manage asynchronous operations and improve system scalability, a Message Broker (e.g., RabbitMQ, Apache Kafka, Redis, ...) will be used in combination with a Task Queue (e.g., Celery).

This is necessary as the SR lifecycle is very complex and involves many operations that can require long execution times [40], such as sending emails, maintaining HDA[3] tickets, integration with external systems, etc. By using a messaging and task queue system, these operations can be executed in the background, without blocking the application's main flow [52].

Provisioning System

It will also be necessary to integrate a provisioning system to execute the scripts related to SRs and those for master data import directly within the clients' networks. This system must be secure, reliable, and scalable, in order to be able to manage a large number of provisioning requests simultaneously.

Elmec already has its own provisioning system called *Provision Prime* [36], which will be integrated with the new SRM system to execute scripts efficiently and securely.

Monitoring and Logging System

To ensure the stability and reliability of the system, a monitoring and logging system will be required. This system will allow tracking system performance, identifying any problems, and analyzing logs to resolve bugs. Tools such as Prometheus, Grafana, New Relic, or other similar tools will be used to implement this system.

Both system metrics (CPU, memory, database usage, API response times, etc.) and application logs (errors, warnings, debug information, etc.) will be tracked, in order to always have a complete view of the system status, and to have the possibility of launching automatic alarms in case of problems (e.g., if the number of responses resulting in 500 (Internal Server Error) in the last hour is $\geq 1\%$).

[3]HDA (Help Desk Advanced) is a web-based ticketing and service management system developed by Zucchetti. It enables organizations to manage tickets, incidents, and support processes through a centralized platform [20].

Containerization and Container Orchestration System

To facilitate system deployment, scalability, and management, a containerization system (Docker) will be used in combination with a container orchestration system (Kubernetes).

This will allow isolating the different system components, simplifying the deployment process, and ensuring greater resilience and scalability of the system, through functionalities such as replicas, chronjobs, automatic scaling of pods, and so on [24].

2.3 Addressing the Identified Challenges

All the challenges described in section 1.3 have been analyzed, addressed, and resolved through a series of targeted interventions. These interventions concerned various aspects of the system, from restructuring the software architecture and revising data management processes to implementing new technologies and development practices. The solutions adopted for each of the identified challenges are detailed below.

2.3.1 Architectural and Technological Challenges

The adoption of an approach based on containerization (with Docker) and orchestration (with Kubernetes, often abbreviated as K8s) forms the foundation for overcoming the current architectural and technological limitations of the SRP system. This transition not only directly addresses issues of modularity, isolation, and dependency management but also paves the way for a more modern and scalable architecture, such as microservices or micro-frontends. Specifically, it aims to solve the problems of fragmentation, dependency complexity, and maintenance difficulties inherited from the previous monolithic architecture and the agent installed at the customer's site.

Containerization with Docker

Docker is a containerization platform that allows developers to package applications and all their dependencies (libraries, configurations, environments) into a single, standardized unit called a container. As illustrated in figure 2.2, these containers are isolated from each other but share the host operating system's kernel, making them extremely lightweight and portable.

The use of Docker resolves the "works on my machine but not in production" problem (known as "dependency hell") [3] and provides a solid foundation for component decoupling. Each component (frontend, backend, support services) will be enclosed in a *Docker container*, and ensures that the execution environment (including libraries, configurations, and dependencies) is identical

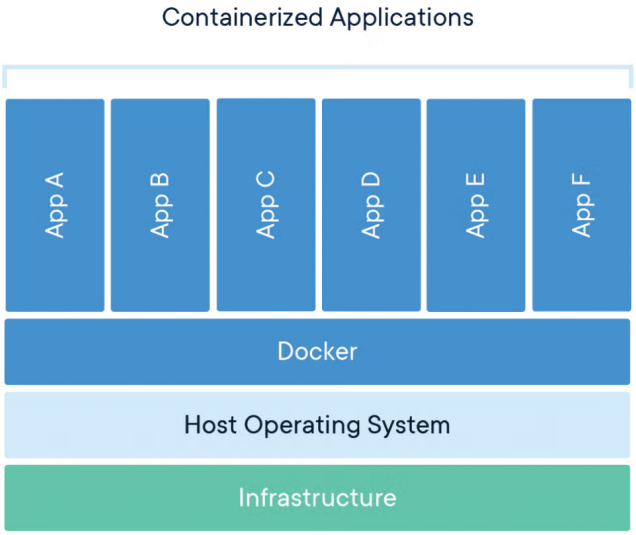

Figure 2.2: Docker Container Architecture (source: [60])

across development, testing, and production [3]. This eliminates errors caused by environmental discrepancies.

Once created, the container can be run anywhere Docker is present (in our case, container orchestration is managed by Kubernetes, which will be discussed in the next subsection), simplifying the deployment process. The system thus becomes extremely more portable, moving away from the strict dependence on specific operating systems and hardware configurations like, in this case, Windows Server and Microsoft SQL Server.

Orchestration with Kubernetes

Kubernetes (K8s) is an open-source system for automated container orchestration, originally developed by Google. Its purpose is to manage entire fleets of Docker containers in a *cluster*, ensuring that applications are always available, scalable, and resilient [24].

If Docker provides the mechanism for creating and distributing containers, Kubernetes is its natural complement, handling their large-scale management (*orchestration*), transforming the infrastructure from a set of single machines into a centrally managed cluster [24].

K8s allows components to scale horizontally based on the workload (via the Horizontal Pod Autoscaler). When demand increases, K8s automatically launches new instances of services (e.g., more pods for the REST backend) and removes them when the load decreases. This is crucial for managing traffic peaks [39].

In parallel, it constantly monitors the status of the containers. If a component fails, K8s automatically restarts it or schedules a new one on a healthy node in the cluster, ensuring high availability and resolving the single-point-of-failure problem introduced by the current dependence on single servers and the agent [39].

Kubernetes also supports advanced deployment strategies (e.g., Rolling Updates), allowing code updates without service interruptions (zero-downtime deployment) [39]. In case of issues, rolling back to the previous version is fast and automated. This mitigates the risk associated with system changes.

Finally, K8s also offers native mechanisms to manage environment variables and sensitive data (such as database credentials or external service authentication tokens) separately from the container code [39]. This centralizes management and increases security, eliminating the numerous hard-coded configurations present in the old system and making the containers more generic.

The adoption of Docker and Kubernetes therefore creates the ideal environment for the subsequent decomposition of the frontend monolith and the re-engineering of the various backend components, topics that will be covered in detail in the following sections.

Monolithinc Frontend Component

At a technological level, the chosen technology did not change compared to the existing system, which is Vue.js. However, Vue.js version 3 was used, which introduces numerous improvements compared to version 2 used in the existing system, including better performance management, a more flexible composition system, and improved support for TypeScript [58]. In fact, in addition to the updated version of Vue.js, it was decided to adopt TypeScript as the development language for the frontend, in order to improve code maintainability and quality [54].

TypeScript Developed in 2012 by Microsoft as free and open-source software (Apache License 2.0) [54], TypeScript extends JavaScript's functionalities by introducing optional static typing. The main advantage is that, unlike JavaScript, where type errors are detected only at runtime, TypeScript allows most errors to be caught already at the *build time* stage. This ensures greater code robustness and a significant reduction in production bugs. For extended projects like this one (the Service Requests component is actually only a part of a much larger customer-dedicated platform called *MyElmec*), static typing significantly improves code maintainability and readability, acting as a form of implicit documentation and allowing for safer and faster refactoring.

To solve the maintainability and scalability problems of the existing monolithic

frontend, it was decided to move all the configuration and customization logic for field behavior (autofill, dynamic fields via external APIs, validation, dependencies, etc.) to the backend (a topic that will be explored in section 2.3.1). In this way, the frontend was transformed into a simple interface for display and user interaction (as it should be), significantly reducing code complexity and facilitating the implementation of new functionalities. In particular, the frontend is now limited to reading a JSON configuration provided by the backend, which describes the form structure, the fields to be displayed, their properties, and validation rules. This was also made possible thanks to the potential offered by Vue.js 3, which allows for the creation of highly dynamic and reusable components based on external data [58].

This led to a transition from a total of 86 different "types" of fields in the existing system to only 10 generic field types:

- Text
- Numeric
- File
- Password
- Checkbox
- Select (with already defined options)
- Select (with API-fetched options)
- Date / DateTime
- UPN (managed specifically due to its peculiarities, as it is composed of two sub-fields and has a particular validation logic)
- ADOU Tree

All the various customization logics (especially the customer-specific ones) thus become transparent to the frontend, which merely interprets the configuration provided by the backend. The system transitioned from a component with approximately 11,000 lines of code to a component with only 1,000 lines of code, resulting in improved maintainability and ease of system extension, and with the possibility of implementing other frontends (e.g., a mobile app) with minimal effort without having to replicate all the complex field management logic.

Overly Generic Backend REST Services

To solve the issues related to the frontend, a deep restructuring of the backend logic was necessary, especially concerning fields, moving from JSON responses like the one shown in Listing 3, to much more detailed and specific responses, which provide the frontend with all the necessary information for the complete rendering of the entire form, such as the one shown in Listing 4.

```
{
    "idAction": 455,
    "idField": 1649,
    "descField": "Manager",
    "type": "ADUser",
    "idCustomer": 42,
    "orderN": 53,
    "mandatory": false
}
```

Listing 3: Example of field data retrieved from legacy SRP System

In addition to modeling all possible properties of a field in a clear and structured way, the new REST services are also responsible for managing all the customization and configuration logic of the fields, with a three-level structure (which will be detailed in the chapter, dedicated to implementation aspects):

1. Fields
2. Actions
3. Customers

Broadly speaking, the idea is that each field (first level "Fields") has a set of generic properties (type, description, options, etc.) that can be further customized based on the specific action (second level "Actions") to which the field belongs (e.g., making it mandatory or not, adding validation rules, autofill logic, dependencies on other fields, etc.). Finally, each action can be further customized per customer (third level "Customers"), allowing the field's behavior to be adapted to the specific needs of each customer (e.g., modifying the available options in a selection field, changing validation rules, etc.), all without affecting the "original" configuration of the underlying level.

This three-level structure allows for a high degree of reusability and modularity, making it possible to define generic fields that can be easily adapted to different situations without having to duplicate code or logic.

Dependency on the SRP Agent

The dependency on the agent installed at the customer site has been completely eliminated thanks to the integration of Provision Prime, an internally

```
{
    "field": {
        "id": 3001,
        "name": "manager",
        "description": "Manager",
        "type": "select",
        "is_sensitive": false,
        "option_source_policy": "EXTERNAL_API",
        "external_api_config": {
            "name": "AD_USERS",
            "base_url": "https://q-api.elmec.com/srp/",
            "endpoint_pattern":
            ↪  "api/customer/VFG/adusers?domain={{domain}}",
            "http_method": "GET",
            "headers_json": {
                "Authorization": "{{TOKEN_TYPE}} {{TOKEN}}"
            },
            "response_mapping_json": {
                "name": "userDesc",
                "value": "user"
            },
            "timeout_seconds": 10,
            "is_searchable": true,
            "search_param_name": "search",
            "root_parent_id": null
        },
        "options": null,
        "multiple_choice": false,
        "file_accept_mimes": null
    },
    "mandatory": false,
    "validation_regex": "^[a-zA-Z0-9._]{5,20}$",
    "validation_regex_message": "Invalid format for Manager field.",
    "autofill_template": null,
    "depends_on": [
        "domain",
        "first_name",
        "last_name"
    ],
    "display_order": 26
}
```

Listing 4: Example of field data returned by the new SRM System

developed provisioning tool written in Go, which manages the execution of Ansible playbooks on remote machines, in this case, the servers within customer networks, transparently to the developer (who will simply configure the playbook and make an API call).

Ansible Ansible is an open-source, agentless IT automation engine used for configuration management, application deployment, intra-service orchestration, and provisioning [4]. It is designed to be simple, powerful, and easy to use, operating over standard SSH for Linux/Unix and WinRM for Windows, thus requiring no special client software (agents) to be installed on the managed nodes [4]. This reliance on existing transport protocols and its human-readable structure, written in YAML, contribute to its minimal learning curve and rapid adoption. The core of Ansible's functionality are *Playbooks*. A Playbook is essentially a blueprint of automation tasks, structured as a YAML file, that describes a set of steps to be executed on specified hosts or groups of hosts (defined in an inventory) [4]. They define the desired state of a system in a clear, declarative manner. Each Playbook consists of one or more plays, and each play is an ordered list of tasks. A task is a call to an Ansible module, which is the unit of code that performs a specific action, such as installing a package, copying a folder, or executing scripts. Playbooks ensure that complex, multi-tier application deployments and job executions are repeatable and reusable, transforming manual, error-prone processes into reliable, version-controlled automation.

In this way, not only is the dependency on the agent, and all the associated logic, eliminated, but all the scripts related to the Service Requests, which previously resided physically inside the customers' Domain Controllers and were not tracked in any way, are now versioned.

The cost of these improvements is represented by the additional logic needed to manage communications with Provision Prime, specifically the retrieval of the target machine. To implement these retrieval logics, which are highly custom for each type of target machine (e.g., Domain Controller, Exchange Server, internal workers, etc.), the *Strategy* design pattern [37] was leveraged. This pattern allows each retrieval logic to be encapsulated in a separate class, making the code more modular, extensible, and maintainable (an example of this pattern is shown in Listing 8 and Listing 15).

As for the script execution logs, which are needed to monitor the status of Service Requests and detect execution errors, an integration with an AWS S3 bucket, a scalable and durable storage service, will be implemented. This way, every time a script is executed on a remote machine, the logs will be uploaded by the DMI (via the Ansible playbook) to the S3 bucket, ensuring centralized and secure access to execution logs regardless of the machine on which they were generated.

2.3.2 Code and Data Management Issues

The lack of versioning and modern debugging capabilities outlined in the existing system is fundamentally addressed by centralizing all disparate logic (previously scattered across Stored Procedures and SSIS jobs) into a unified, version-controlled software project integrated with a robust CI/CD pipeline. By adopting Git as the standard Version Control System (VCS), all business logic is now treated as application code. This immediately solves the versioning problem: every change is tracked, attributed to an author, and can be reverted, enabling seamless collaboration and a full audit trail. Moreover, moving the core business logic out of the database and into a dedicated backend service (Django REST Framework) written in a modern language (Python) drastically improves debuggability, allowing developers to use powerful, standardized IDE tools to set breakpoints, inspect variables, and step through code without impacting the production database.

This strategy is completed by integrating several specific CI/CD pipelines (a topic that will be explored in depth in Section 3.1.1) tailored for each environment branch (`feature`, `devel`, `quality`, `main`). This setup automates the deployment process: code changes are first pushed to a dedicated task branch, reviewed, and then merged into the `devel` environment. Upon passing, the code is merged to the `quality` environment, and finally to `main` for production release. This process ensures that no logic is directly modified in a production environment, eliminating the single-point-of-failure inherent in direct database and server access. Furthermore, replacing the inaccessible, unversioned, and tightly coupled SSIS Jobs with modern, containerized async tasks (managed as code within the Git repository) breaks the strong infrastructure coupling, enhancing accessibility, facilitating collaboration, and making maintenance and debugging a safe, environment-specific activity.

2.3.3 Database Schema and Data Integrity Issues

The issues related to database design and data management were addressed through a series of interventions aimed at improving data integrity, consistency, and maintainability. These interventions include a complete revision of the database schema to eliminate redundancies and inconsistencies, the implementation of stricter integrity constraints, and the adoption of clear and consistent naming conventions for tables and fields.

The system transitioned from using Microsoft SQL Server as the Database Management System (DBMS) to PostgreSQL, an open-source DBMS known for its robustness, scalability, and compliance with SQL standards. PostgreSQL offers advanced features such as support for complex data types, full ACID transactions, and an extensibility system that allows new functionalities to be added via extensions [49]. This choice not only improves the system's performance and reliability but also facilitates data management and the

implementation of database design best practices.

Lack of Data Integrity and Consistency

To resolve data integrity and consistency issues, stricter referential integrity constraints were implemented using well-defined primary and foreign keys, also specifying behaviors in case of deletion (`ON DELETE CASCADE` / `SET_NULL`, etc...).

Specifically, for the deletion of "core" entities (such as customers, actions, or fields), the `CASCADE` behavior was chosen for "child" entities, which are consequently deleted.

For secondary entities and/or those that can be null (such as ticket categorization information, action categories and groups, order information associated with tickets, etc...), the `SET_NULL` behavior was chosen to preserve the history of operations and associated data, preventing the loss of more important information.

These constraints ensure that relationships between tables are correctly maintained, preventing the creation of orphaned or inconsistent data.

Data Redundancy

New relationships were created to maximize data reuse and reduce redundancy, while ensuring referential integrity through the use of well-defined primary and foreign keys.

These new entities primarily model concepts that were previously duplicated across multiple tables, such as cost centers (CDC, *"centro di costo"*), service codes, and ticket classification and category. This significantly reduced the amount of redundant data, improving storage efficiency and database maintainability.

Unclear and Inconsistent Naming Conventions

Now, all tables and their attributes follow standardized naming conventions (*snake_case*), improving the readability and maintainability of the database schema.

For "core" relationship names, the plural form was chosen (e.g., `actions`, `service_requests`, `customers`, etc...), while for "child" relationships, the singular name of the "parent" relationship followed by the specific name in the plural was chosen (e.g., `action_fields`, `customer_actions`, etc...).

2.3.4 Maintenance and Operational Challenges

With the new improvements introduced in this chapter concerning architecture, technology, the completely redesigned catalog and configuration management, and the new execution flow, many of the previously encountered maintenance and operational problems have been resolved. The standardization of development and deployment processes, the adoption of DevOps practices, and the implementation of a more modern and scalable infrastructure have led to a significant reduction in downtime, while improving the system's overall reliability. Furthermore, the adoption of advanced monitoring and logging tools allows for rapid identification and resolution of any issues, ensuring smooth and continuous system operation.

These improvements not only facilitate daily operations but also long-term planning, enabling the system to adapt easily to future needs and challenges.

Chapter 3

Implementation of a modular and scalable solution

While the previous chapter described in detail the requirements and the new architectural design for the new ServiceRequestManager (SRM) system, proposing targeted solutions to overcome the various technological and structural limitations of the legacy SRP system, this chapter marks the transition from the theoretical plan to practical realization, focusing on the concrete implementation of a modular and scalable solution.

The primary focus of this implementation lies in how the critical challenges described in the previous chapters were resolved. Specifically, it will be highlighted how the centralization of field management and configuration logic (and more generally of the catalog) on the backend eliminated the monolithic frontend and enabled a highly reusable three-level configuration system. The integration of the new execution and retrieval flow for master data through a message queue system will also be illustrated, which is essential for eliminating the dependency on the obsolete SRP Agent.

3.1 Automation of the Development Lifecycle through CI/CD Integration

The project implements a comprehensive CI/CD pipeline that automates code quality verification, security scanning, testing, building, and deployment across three distinct environments: `devel`, `quality`, and `production`. The Git workflow requires feature branches to merge sequentially through these environments, ensuring progressive validation before production release.

3.1.1 Pipeline Architecture

The CI/CD pipeline is organized into three distinct workflows based on branch type, as illustrated in Figures 3.1, 3.2, and 3.3.

Feature Branch Pipeline

For feature branches, the pipeline executes automated checks for code quality, security, and compliance without building or deploying the application. The

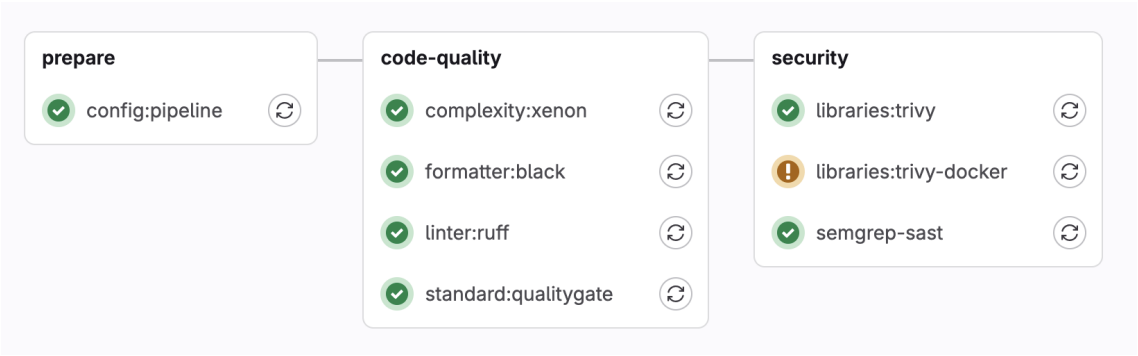

Figure 3.1: Feature branch pipeline workflow with quality and security checks

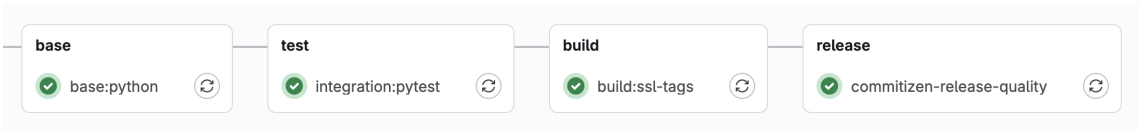

Figure 3.2: Live environment pipeline workflow with build, test, and release stages

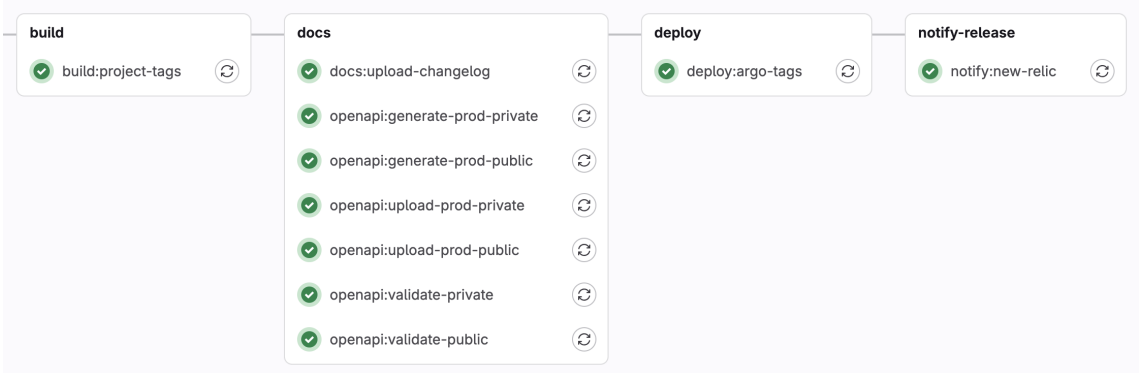

Figure 3.3: Tag-based deployment pipeline with documentation and ArgoCD integration

`complexity:xenon` stage measures code complexity through the Xenon static analyzer [62], identifying functions that might be difficult to maintain. The `formatter:black` stage verifies Python code compliance with PEP-8 conventions [33], while `linter:ruff` performs static analysis to detect errors and unsafe constructs.

Security scanning occurs through two Trivy stages: `libraries:trivy` identifies known vulnerabilities in dependencies against CVE databases [53], while `libraries:trivy-docker` scans deployment files and Kubernetes manifests for misconfigurations. The `SAST` stage uses Semgrep to detect security vulnerabilities such as SQL injection and XSS without code execution [43].

Live Environment Pipeline

For main branches (`devel`, `quality`, `master`), the pipeline includes additional stages shown in Figure 3.2. The `base:python` stage builds the base Docker image containing Python dependencies, implementing layer separation to optimize build times through intelligent cache usage. The `test:pytest` stage executes the automated test suite against PostgreSQL 16, enforcing an 80% code coverage threshold.

The `commitizen-release` stage manages automatic versioning using Commitizen, which analyzes commit history to determine version numbers according to Semantic Versioning and Conventional Commits conventions. This automated approach eliminates manual version management and ensures consistency in the release process.

Tag-Based Deployment Pipeline

When a semantic version tag is created, the pipeline triggers the workflow shown in Figure 3.3. The `build:project-tags` stage assembles the final Docker image from previously built layers and publishes it to the company registry. Documentation stages generate OpenAPI/Swagger specifications using Django Spectacular and publish changelogs to the internal documentation platform.

The `deploy:argo-tags` stages manage automatic deployment through ArgoCD [6], a GitOps tool that synchronizes Kubernetes cluster state with Git-declared configurations. Listing 5 shows the deployment trigger structure.

The `notify:new-relic` stage registers deployments in New Relic APM, creating markers that correlate software versions with performance metrics and enabling rapid identification of deployment-related issues.

```yaml
deploy:argo-tags:
  stage: deploy
  only:
    - /^\d+\.\d+\.\d+$/  # Semantic version tags only
  script:
    - candymand deploy --namespace servicerequestmanager
                       --version $CI_COMMIT_TAG
```

Listing 5: ArgoCD Deployment Configuration for GitOps-based Releases

3.2 Implementation of the Deployment Architecture on Kubernetes

The SRM deployment architecture on Kubernetes ensures scalability, resilience, and separation of responsibilities through distinct deployments optimized for specific system functions. The high-level structure shown in Listing 6 defines the namespace, service accounts, deployments, and scheduled jobs.

```yaml
namespace: servicerequestmanager
serviceAccount:
  create: TRUE
deployments:
  - name: static    # Nginx for static content
  - name: backend   # Django API + Celery workers
jobs:
  - name: sync-srp-catalog        # Daily at 01:00
  - name: sync-srp-sr-history     # Daily at 02:00
  - name: import-directory-data   # Daily at 02:00
```

Listing 6: High-Level Kubernetes Deployment Structure for SRM

3.2.1 Backend Application Deployment

The `backend` deployment constitutes the architecture core, managing application logic through multi-container organization. The deployment integrates HashiCorp Vault [57] for secrets management, automatically injecting credentials into the pod filesystem at `/vault/secrets/secrets` without exposing them in environment variables.

The `api` container runs Django through Gunicorn with Burstable QoS [35] (400m-800m CPU, 1000Mi-1200Mi memory), while the `celery` container executes asynchronous workers with identical resource allocation. Listing 7 shows the worker configuration.

The deployment exposes two services: `api` for internal cluster access and `api-ext` for public access through `api.elmec.com/servicerequestmanager/`, with path rewriting middleware removing the prefix. CORS policies permit cross-origin requests supporting third-party integrations.

```
- name: celery
  image: "docker.elmec.com/.../servicerequestmanager:VERSION"
  command: "/bin/sh"
  args:
    - -c
    - . /vault/secrets/secrets; cd servicerequestmanager;
      celery -A servicerequestmanager worker --loglevel=info
             --pool threads
  envs:
    - name: CELERY_QUEUE
      value: "servicerequestmanager_production"
    # ...
```

Listing 7: Celery Worker Container Configuration in Kubernetes

3.2.2 Scheduled Synchronization Jobs

Kubernetes CronJobs maintain system alignment during the SRP-SRM co-existence period. The `sync-srp-catalog` job executes daily at 01:00 AM, importing catalog configurations from the SRP SQL Server database. The `sync-srp-sr-history` job runs at 02:00 AM, synchronizing service request execution history. The `import-directory-data` job triggers Ansible playbooks for master data export from customer Domain Controllers.

3.3 Backend Service Implementation

The backend service is the most critical component of the SRM architecture, responsible for managing application logic, processing service requests, and integrating with external systems. Its implementation was designed to ensure scalability, resilience, ease of maintenance, and a high degree of customization to accommodate the specific needs of clients.

3.3.1 Technological Stack

The backend leverages several key technologies that form the foundation of the system architecture.

Core Framework

The backend is built on *Django* 5.2 with *Django REST Framework* (DRF) 3.16, implementing a Model-View-Controller pattern where Django models handle data persistence, DRF viewsets manage business logic and request processing, and serializers handle data validation and transformation [15][16]. This architecture was selected over alternatives like Go with GORM primarily due to development velocity considerations: Django's comprehensive ecosystem of built-in components (ORM, authentication, admin interface) significantly reduces time-to-market compared to implementing equivalent functionality in

Go. While Go offers superior raw performance, the anticipated load characteristics of SRM (primarily CRUD operations and orchestration) do not justify trading ecosystem maturity for performance gains.

Asynchronous Processing

Celery with *Redis* as the message broker implements the asynchronous task queue for service request execution and scheduled operations [9]. This producer-consumer architecture enables Django to publish tasks to Redis while dedicated worker processes consume and execute them independently, preventing long-running operations from blocking API responses. Celery was configured with thread-based workers optimized for I/O-bound operations (API calls, database queries, remote script execution coordination) rather than CPU-intensive processing.

Data Layer

PostgreSQL serves as the primary data store, selected for its robust ACID transaction guarantees, sophisticated query planning, and support for partial indexes and Common Table Expressions (CTEs) essential for hierarchical catalog queries. The application connects through *PgBouncer*, a connection pooler that maintains persistent database connections and enables horizontal scaling of application pods without exhausting database connection limits [11].

Memcached implements distributed in-memory caching [28], specifically for responses from external financial and administrative systems (contracts, cost centers, project codes, server informations, ...). These external services exhibit high latency but return data with temporal stability, making them ideal candidates for caching with configurable TTL values.

Remote Execution

Ansible orchestrates service request execution on customer infrastructure through agentless SSH/WinRM connections [4], eliminating the dependency on the legacy SRP Agent. Custom playbooks encapsulate provisioning workflows that currently wrap existing PowerShell scripts with a migration path toward native Ansible modules.

Object Storage

S3-compatible object storage (*Ceph RGW* via MinIO API) manages master data exports and execution artifacts. Master data is exported to JSON in S3, then materialized in PostgreSQL for efficient querying, achieving sub-second response times compared to multi-second LDAP query latencies.

3.3.2 Application Architecture

The Django project is organized into six specialized applications following separation of concerns principles: `catalog/` manages the three-level configuration hierarchy; `integration/` handles external system integrations; `execution/` implements service request lifecycle management; `monitoring/` provides health checks; `directory_data/` manages master data import; and `common/` contains shared utilities.

3.3.3 Catalog Management System

The catalog implements a three-level configuration hierarchy that eliminates the monolithic field duplication of the legacy system while providing granular customization capabilities.

Configuration Hierarchy

As shown in Figure 3.4, the architecture consists of three distinct levels:

- Level 1 defines global `Action` and `Field` models with default configurations.
- Level 2 provides `CustomerAction` for customer-specific action modifications.
- Level 3 offers `CustomerActionField` for granular field-level customer-specific customization.

Configuration resolution follows strict precedence: `CustomerActionField` overrides `ActionField`, which overrides `Field`, while `CustomerAction` overrides `Action`. Listing 8 shows the implementation of the target type enumeration that determines execution routing.

```python
class ActionTargetTypes(Enum):
    DOMAIN_DC = "From SR Domain (Domain Controller)"
    DOMAIN_EXC = "From SR Domain (Exchange Server)"
    INTERNAL_WORKER = "Internal Worker"
    SRP = "SRP Agent (Domain Controller)"

    def get_class(self) -> BaseActionTargetType | None:
        match self:
            case ActionTargetTypes.DOMAIN_DC:
                return DCFromDomainTargetType()
            # ...
```

Listing 8: Action Target Type Enumeration Implementation

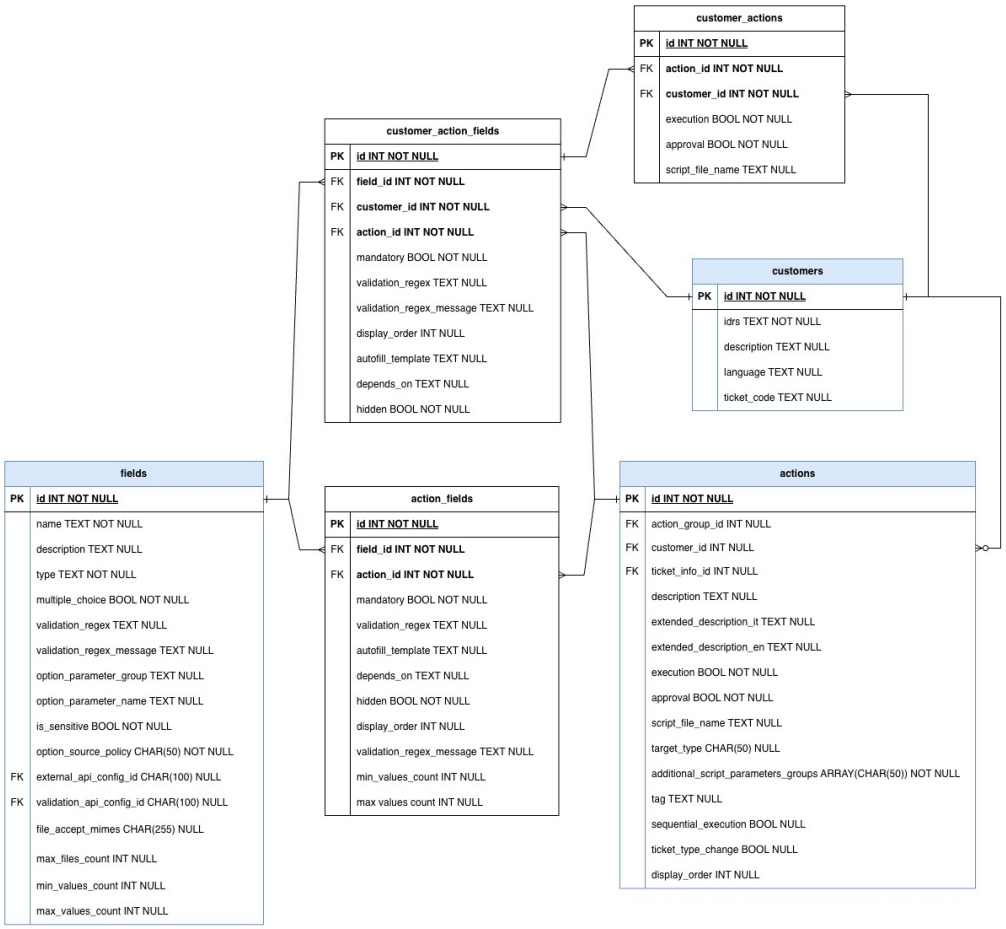

Figure 3.4: ER Diagram of the Three-Level Catalog Configuration Hierarchy

Key Architectural Patterns

Service Block-Based Action Association: Customer access to actions is determined through service blocks derived from contract information rather than granular one-to-one associations, significantly reducing administrative overhead.

Field Type System: The catalog implements 14 distinct field types including simple types (text, password, email), complex types (text lists, select dropdowns, UPN inputs), and specialized validation for each type.

Templating System: Jinja2 templating enables dynamic behavior throughout the configuration layer. Field autofill templates define patterns for prepopulating values based on other fields, as shown in Listing 9.

```
# Template example for UPN autofill
autofill_template = "{{ first_name }}.{{ last_name }}"
# Renders to: "john.doe" when first_name="john", last_name="doe"

# API endpoint pattern with templating
endpoint_pattern = "/api/customers/{{ idrs }}/adusers?domain={{ domain }}"
# ...
```

Listing 9: Jinja2 Templating Examples for Dynamic Field Configuration

3.3.4 Core Execution Workflows

Field Validation

The validation system, centralized in `ServiceRequestValidationMixin`, implements comprehensive pre-submission validation. The architecture separates global validators (operating on complete field sets) from field validators (examining individual field-value pairs). Listing 10 demonstrates the regex validation implementation.

```
def validate_field_value_regex(
    field: Field, value: str | None, **_kwargs
) -> None:
    if field.validation_regex and value:
        if not re.fullmatch(field.validation_regex, value):
            raise serializers.ValidationError({
                "fields_validation": [
                    f"The value '{value}' does not match pattern"
                ]
            })
```

Listing 10: Field Validation Using Regex Patterns

Validation progresses through multiple layers: syntactic validation via regex patterns, configuration-based validation, option validation for select fields, and specialized validation for external endpoints. The framework implements

error aggregation, continuing validation after detecting failures to accumulate all errors before returning consolidated responses.

Asynchronous Execution Flow

The execution workflow implements event-driven task orchestration through Celery. The flow implements a state machine (NEW, WAIT4APPROVAL, APPROVED, PENDING, EXECUTED, SUSPENDED, ERROR, REJECTED) coordinated through six specialized tasks. Listing 11 shows the structure of a typical Celery task.

```python
@shared_task()
def manage_new_service_request_and_ticket(sr_id: int):
    sr = ServiceRequest.objects.get(id=sr_id)

    if sr.action.requires_approval:
        sr.status = "WAIT4APPROVAL"
        sr.save()
    else:
        sr.status = "APPROVED"
        # ...
        execute_service_request.delay(sr_id)
```

Listing 11: Celery Task for Service Request State Management

Script execution occurs through Ansible playbooks implementing a five-phase pattern: preparation, file transfer, execution, logging, and cleanup. This maintains transparency for script developers while providing centralized execution logging.

Master Data Import Pipeline

The master data import subsystem eliminates real-time LDAP query latency through a scheduled export-materialize-query pattern. Listing 12 demonstrates the differential synchronization logic.

3.3.5 Observability and Monitoring

The system implements comprehensive observability through multiple layers: New Relic APM for transaction monitoring, structured logging with automatic New Relic Logs integration, health checks via the `/ht/` endpoint, and Prometheus metrics exposure. The `@monitor_job` decorator wraps scheduled tasks as shown in Listing 13.

3.4 Frontend Application Implementation

The SRM frontend comprises two Vue.js 3 applications: an end-user service request creation interface and an administrative configuration platform.

```python
@transaction.atomic
def process_users_for_customer(customer_id: int, user_data_list):
    logon_names_from_file = set(
        (ud["domain"], ud["logon_name"]) for ud in user_data_list
    )
    existing_users = {
        (user.domain, user.logon_name): user
        for user in AdUser.objects.filter(customer_id=customer_id)
    }

    users_to_create = []
    users_to_update = []
    users_to_delete = []
    # ... differential logic

    if users_to_create:
        AdUser.objects.bulk_create(users_to_create, batch_size=1000)
    if users_to_update:
        AdUser.objects.bulk_update(users_to_update,
            fields=["domain", "distinguished_name"], batch_size=1000)
```

Listing 12: Differential Synchronization for Master Data Import

```python
def monitor_job(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        application = newrelic.agent.register_application()
        start_time = datetime.now()

        try:
            return_val = func(*args, **kwargs)
            # Send completion event to New Relic
        except Exception as err:
            # Send error event with traceback
            raise
        finally:
            end_time = datetime.now()
            # Log duration and status
    return wrapper
```

Listing 13: Job Monitoring Decorator for Observability

3.4.1 Technology Stack

Both applications leverage Vue 3 Composition API with TypeScript for type safety [58]. State management is implemented through Pinia [34], and the component architecture utilizes a custom design system library (ElmecUIX).

3.4.2 Service Request Creation Interface

The end-user interface implements a dynamic form builder rendering forms based on backend API responses. A single GET request retrieves complete action definitions including resolved field configurations. Listing 14 shows the client-side validation rules generation.

```
const formRules = computed(() => {
  const rules: Record<string, any[]> = {}

  for (const af of action.value.action_fields) {
    const fieldName = af.field.name
    if (!af.mandatory && !af.validation_regex) continue

    rules[fieldName] = []
    if (af.mandatory) {
      rules[fieldName].push({
        required: true,
        message: t("general.required")
      })
    }
    if (af.validation_regex) {
      rules[fieldName].push({
        validator: (rule, value, callback) => {
          const regex = new RegExp(af.validation_regex)
          if (!regex.test(value)) {
            callback(new Error(af.validation_regex_message))
          } else {
            callback()
          }
        }
      })
    }
  }
  return rules
})
```

Listing 14: Dynamic Form Validation Rules in Vue.js

The component architecture separates concerns through parent-child relationships. Field dependency resolution is implemented through reactive watchers monitoring form data changes. External API option caching minimizes redundant backend calls through URL tracking and result storage.

3.4.3 Administrative Configuration Platform

The administrative interface (in design phase) will provide CRUD operations for managing catalog hierarchy components with form-based editors, visual representation of override relationships, and preview functionality enabling administrators to test configurations before deployment.

The frontend architecture achieves dynamic, configuration-driven interfaces that adapt to catalog changes without code modifications, enabling rapid iteration on service request workflows.

Chapter 4

Coexistence and Gradual Migration Strategy

The transition from the legacy SRP system to the new ServiceRequestManager architecture represents a substantial technical and operational challenge that requires careful planning to minimize business disruption while ensuring data integrity and system reliability. Rather than implementing a disruptive "big bang" migration, the project adopts a phased coexistence strategy that enables gradual migration of customers and service request types while maintaining full operational continuity.

4.1 Migration Strategy Overview

The migration architecture implements a dual-flow execution model where both SRP and SRM systems operate simultaneously, with individual service request actions configured to route through either the legacy or modern execution pipeline based on the `target_type` attribute of the Action model, as shown in Listing 15.

```
class ActionTargetTypes(Enum):
    DOMAIN_DC = "From SR Domain (Domain Controller)"
    DOMAIN_EXC = "From SR Domain (Exchange Server)"
    INTERNAL_WORKER = "Internal Worker"
    INTERNAL_API = "API Call"
    SRP = "SRP Agent (Domain Controller)"  # Legacy flow
```

Listing 15: Action Target Type Enumeration for Dual-Flow Routing

Actions configured with `target_type = SRP` continue routing through the legacy SRP Agent execution mechanism, preserving the existing workflow without modification. Actions configured with any other target type utilize the new Ansible-based execution pipeline through Provision Prime, benefiting from improved logging, error handling, and state management capabilities introduced in SRM.

This per-action migration control provides several strategic advantages: it enables pilot deployments on specific customers or action types; it maintains business continuity by ensuring that any discovered issues affect only the subset of actions migrated to the new flow; it supports progressive learning;

and it provides fallback capability, as actions can be reverted to SRP execution if critical issues are discovered.

The primary architectural trade-off is that the catalog configuration must be maintained in the SRP database during the coexistence period, as SRP lacks the sophisticated configuration features introduced in SRM (three-level hierarchy, external API configurations, template-based autofill). The SRM system operates as a read-only consumer of catalog data from SRP, applying its advanced configuration features as an overlay on the imported base catalog.

4.2 Catalog Synchronization from SRP to SRM

The catalog synchronization mechanism implements a unidirectional data flow where SRP serves as the authoritative source of catalog definitions, with SRM periodically importing and enriching this data. This synchronization is executed through the `sync_srp_catalog` Django management command, scheduled as a Kubernetes CronJob for daily execution at 01:00 AM, as shown in Listing 16.

```
@monitor_job
def handle(self, *args, **kwargs):
    with disable_auditlog(), pyodbc.connect(SRP_CONN_STR) as conn:
        cursor = conn.cursor()
        sync_action_types(cursor)
        sync_action_categories(cursor)
        sync_action_groups(cursor)
        sync_customers(cursor)
        sync_actions(cursor)
        sync_config_parameters(cursor)
        sync_fields(cursor)
        # ...
        reset_sequences(app_label="catalog")
```

Listing 16: Catalog Synchronization Job Structure

4.2.1 Synchronization Architecture

The synchronization process establishes a direct ODBC connection to the SRP SQL Server database and executes a series of import operations that reconstruct the complete catalog hierarchy in the SRM PostgreSQL database. The process imports entities in a carefully ordered sequence that respects foreign key dependencies: action types → categories → groups → customers → actions → configuration parameters → fields → customer-specific customizations.

The synchronization executes with audit logging disabled to prevent creation

of thousands of audit log entries for bulk import operations, improving performance and reducing database bloat. Each synchronization function implements a consistent pattern: query the source table in SRP, construct Django model instances for new or modified records, and execute bulk create/update operations.

4.2.2 Differential Synchronization Logic

The synchronization functions implement sophisticated differential logic that minimizes database operations by detecting actual changes rather than blindly recreating all data. Listing 17 demonstrates the pattern used across synchronization functions.

```
records = [dict(zip(columns, row)) for row in cursor.fetchall()]
source_ids = {int(r["IDCategory"]) for r in records}

existing_categories = {
    ac.id: ac for ac in ActionCategory.objects.filter(
        id__in=source_ids
    )
}

to_create = []
to_update = []

for record in records:
    category_id = int(record["IDCategory"])
    if category_id in existing_categories:
        obj = existing_categories[category_id]
        obj.description = record["DescCategory"]
        # ...
        to_update.append(obj)
    else:
        to_create.append(ActionCategory(
            id=category_id,
            description=record["DescCategory"]
            # ...
        ))

if to_create:
    ActionCategory.objects.bulk_create(to_create)
if to_update:
    ActionCategory.objects.bulk_update(to_update,
        ['description', 'display_order'])
```

Listing 17: Differential Synchronization Pattern with Bulk Operations

This approach constructs in-memory dictionaries of existing records keyed by primary key, then iterates through source records to segregate them into create and update lists. Bulk operations execute with batch sizes appropriate for PostgreSQL, significantly outperforming individual save operations. An initial full synchronization strategy that deleted and recreated all records at

each execution proved inefficient with SRP's large data volumes, resulting in execution times around 2 hours. The current differential logic significantly reduces database load, bringing average execution times down to approximately 20 seconds.

4.2.3 Post-Import Enrichment

Following the base catalog import from SRP, the synchronization process executes several enrichment operations that apply SRM-specific configurations: API configuration synthesis creates `ExternalAPIConfiguration` records defining option sources for select fields; field duplication resolution identifies and resolves scenarios where multiple fields with identical names exist within the same action scope; HDA field configuration synthesizes metadata for external fields and classification fields; and customer-specific customizations translate SRP configurations into SRM's three-level hierarchy.

The synchronization concludes by resetting PostgreSQL auto-increment sequences to prevent primary key conflicts on subsequent manual record creation.

4.3 Execution History Synchronization

While catalog synchronization flows unidirectionally from SRP to SRM, execution history must be merged from both systems to provide unified visibility. The `sync_srp_sr_history` command implements this bidirectional synchronization, scheduled to execute at 02:00 AM (one hour after catalog sync).

The execution history synchronization imports three primary entity types: service request headers, field values, and execution logs. The synchronization is filtered by a configurable date threshold to avoid importing the complete historical archive, defaulting to the first day of the current month, as shown in Listing 18.

```
start_date = os.getenv(
    "SRP_SR_SYNC_START_DATE",
    datetime.date.today().replace(day=1).strftime("%Y-%m-%d")
)
cursor.execute(
    f"SELECT * FROM TBSR_Headers WHERE Date > '{start_date}'"
)
# ... process service requests
sync_service_requests(cursor)
sync_service_request_fields(cursor)
sync_service_request_logs(cursor)
```

Listing 18: Execution History Synchronization with Date Filtering

Service request field values imported from SRP must respect SRM's enhanced

security model where sensitive fields are encrypted at rest. The synchronization implements transparent encryption during import, checking each field's sensitivity flag and applying encryption to values from sensitive fields. Execution logs stored in SRP use a numeric type code system incompatible with SRM's enumerated log types, requiring a mapping table that translates SRP codes to SRM equivalents (Creation, Approval, Execution Started, Execution Completed, etc.).

4.4 Dual Execution Flow Management

The coexistence architecture's core mechanism is the dynamic routing of service request execution based on action configuration. When a service request is created through the SRM API, the system evaluates the action's `target_type` to determine whether to invoke the new or legacy execution pipeline, as illustrated in Listing 19.

```
is_srp_flow = action.target_type == ActionTargetTypes.SRP.name

# Create HDA ticket (common to both flows)
response_ticket = mysupport.create_ticket(
    codicli=codicli, idrs=customer.idrs, body=body
)
ticket_id = response_ticket.get("ID")

if is_srp_flow:
    # Legacy flow: SRP will create the SR entity
    logger.info(f"SRP flow detected. Ticket {ticket_id} created.")
    return ServiceRequest(
        action=action, customer=customer,
        ticket_reference=ticket_id
    )
else:
    # Modern flow: SRM creates SR and triggers Celery tasks
    validated_data["ticket_reference"] = ticket_id
    service_request = super().create(validated_data)
    # ... trigger async execution
    return service_request
```

Listing 19: Dual Execution Flow Routing Logic

For actions configured with `target_type = SRP`, the serializer creates the HDA ticket but returns a non-persisted ServiceRequest object. The actual service request record will be created by SRP's legacy REST API when HDA invokes it, then synchronized into SRM's database by the nightly job. For actions with any non-SRP target type, the ServiceRequest record is persisted immediately with NEW status, and the asynchronous Celery task chain is initiated.

The dual-flow architecture remains transparent to frontend applications, which interact exclusively with SRM APIs regardless of the underlying execution

mechanism, ensuring that frontend applications require no modifications as actions are gradually migrated.

4.5 Pilot Strategy and Progressive Rollout

The migration strategy implements a risk-managed progression that validates the new architecture incrementally before full deployment, organized into four distinct phases.

4.5.1 Phase 1: Internal Pilot

The initial migration phase configures a small subset of actions used exclusively by internal IT operations teams to execute through the SRM pipeline. These actions are selected based on low execution volume, non-critical business processes, and comprehensive logging requirements. Internal teams provide direct feedback on execution behavior, enabling rapid iteration on issues discovered during real-world usage.

4.5.2 Phase 2: Customer Pilot

Following successful internal validation, the migration expands to a carefully selected cohort of pilot customers chosen based on technical sophistication, diverse action usage patterns, and strong relationship with the company. Each pilot customer has a subset of their actions migrated to SRM execution, typically starting with low-risk operations before progressing to more critical provisioning workflows.

The pilot phase implements enhanced monitoring including dedicated Grafana dashboards, daily review meetings, and direct communication channels with pilot customers. Issues discovered during pilot are addressed through hotfix deployments, with actions reverted to SRP execution if critical problems cannot be resolved quickly.

4.5.3 Phase 3: Progressive Expansion

Successful pilot completion triggers progressive expansion where additional customer cohorts are migrated on a weekly cadence. Each expansion wave includes a defined set of customers and actions, with execution metrics monitored for anomalies before proceeding to the next wave. The expansion velocity is dynamically adjusted based on observed success rates: high success rates (>99%) enable acceleration, while elevated error rates trigger pause for investigation.

The rollout maintains a progressive migration of action types from simple to complex: read-only information queries migrate first, followed by non-

destructive provisioning operations, then critical operations, and finally complex multi-step workflows.

4.5.4 Phase 4: SRP Decommissioning

Migration completion occurs when all customers and actions have successfully transitioned to SRM execution, validated through sustained periods (30+ days) of stable operation without SRP fallback. The SRP system transitions to read-only mode, the catalog synchronization job is deactivated, and finally the SRP infrastructure is decommissioned after a retention period ensuring all audit and compliance requirements are satisfied.

4.6 Data Integrity and Consistency Mechanisms

The coexistence architecture implements several mechanisms to maintain data integrity and consistency during the migration period.

4.6.1 Transactional Boundaries and Idempotency

Synchronization operations execute within database transaction contexts ensuring atomicity: either all records in a synchronization batch commit successfully, or none do. The synchronization logic is designed for idempotency, enabling safe re-execution in case of failures. Each sync function can be invoked multiple times with identical results, as record identification is based on deterministic primary keys from SRP.

4.6.2 Referential Integrity and Audit Trail

Foreign key relationships are validated before record creation to prevent orphaned references. The synchronization order ensures parent entities exist before children, while optional relationships are nulled when references cannot be resolved. While audit logging is disabled during bulk synchronization for performance, the synchronization jobs themselves are comprehensively logged through the monitoring infrastructure, with execution metrics recorded in New Relic.

4.7 Operational Considerations and Migration Benefits

The coexistence strategy introduces several operational challenges that require careful management during the migration period.

4.7.1 Operational Challenges

The primary challenges include: dual maintenance burden requiring catalog changes in SRP that propagate through synchronization; synchronization latency introducing up to 24-hour delays between SRP changes and SRM visibility; execution history fragmentation requiring staff training to query SRM exclusively; and performance considerations necessitating scheduled execution during low-traffic periods.

4.7.2 Risk Mitigation and Benefits

The coexistence and gradual migration strategy provides a pragmatic path to modernization while maintaining operational continuity. The dual-flow architecture enables incremental validation of the new system in production environments, building confidence through measured progression rather than high-risk wholesale replacement. The phased rollout provides operations teams with extended periods to develop familiarity with the new system architecture, monitoring tools, and troubleshooting procedures.

The coexistence strategy eliminates migration-related service disruptions by maintaining the legacy system as a fully functional fallback throughout the transition period. This approach ensures that service level agreements are maintained throughout the migration, avoiding the business impact of failed "big bang" migrations. The gradual migration provides real-world validation of architectural decisions under production loads, enabling refinement based on operational feedback before full deployment.

While the strategy introduces operational complexity through the need for synchronization and dual-system maintenance, these costs are justified by the substantial risk reduction and business continuity assurance provided by the gradual approach. The coexistence period represents a necessary investment in migration safety that protects both the company and its customers from the substantial risks inherent in large-scale system replacements.

Chapter 5

Recommendation System through LLM and Retrieval Augmented Generation

The ServiceRequestManager catalog, while providing comprehensive coverage of available service request types through its three-level configuration hierarchy, presents end users with a potentially overwhelming interface containing hundreds of distinct actions organized across multiple categories and groups. Traditional hierarchical navigation requires users to understand the organizational taxonomy of IT services, navigate through nested category structures, and select from long lists of similarly-named actions, a process that introduces friction in the user experience and may result in suboptimal action selection or abandonment of the request creation workflow.

This chapter presents the design and implementation of an intelligent recommendation system that leverages Large Language Models (LLMs) and Retrieval Augmented Generation (RAG) to enable natural language-based action discovery. The system accepts free-form textual descriptions of user intent (e.g., "I need to reset the password for a user account") and automatically identifies the most relevant catalog actions, providing ranked recommendations with confidence scores and explanatory reasoning that enables users to validate the system's suggestions before proceeding to request creation.

5.1 Motivation and Problem Statement

The development of a recommendation system for SRM addresses several fundamental challenges inherent in enterprise service request management, where the complexity of service catalogs and the diversity of user technical expertise create barriers to efficient self-service.

5.1.1 Limitations of Traditional Catalog Navigation

The SRM catalog exposes over 300 distinct actions organized hierarchically through action types (System, Network, Personal), categories (Active Directory, Exchange, Office 365), and groups (User Management, Group Management, Mailbox Management). While this organizational structure provides

logical grouping for administrative purposes and supports scalable catalog management, it presents several usability challenges for end users:

Cognitive Load of Taxonomy Navigation: Users must understand the IT service taxonomy to locate relevant actions. A user seeking to create a mailbox must first recognize that this falls under the "Personal" type, then navigate to the "Exchange" category, and finally locate the "Mailbox Management" group. This multi-level navigation assumes familiarity with IT organizational structures that many business users lack, particularly in heterogeneous environments where similar services may be organized differently across customers.

Ambiguity in Action Descriptions: Many actions have similar or overlapping descriptions that are distinguishable only through careful reading of extended descriptions or examination of required fields. For example, "Create User Account," "Create User with Mailbox," and "Create User with Office 365 License" represent distinct actions with different provisioning workflows, but their descriptions may appear similar during rapid scanning. Users unfamiliar with the technical distinctions between these options may select suboptimal actions, leading to subsequent corrections or additional requests.

Discovery of Relevant Actions: The hierarchical organization assumes users know which category contains the service they need. In reality, users often describe needs in business terms ("I need the new employee to access email") rather than technical terms ("Create Exchange mailbox"), making it difficult to identify the appropriate category. The lack of cross-category search capabilities exacerbates this issue, as users must exhaustively browse multiple categories to locate relevant actions.

Evolution of Catalog Complexity: As the SRM catalog continues to grow through addition of new actions for emerging services and customer-specific customizations, the navigation challenge intensifies. The catalog's flexibility, which is a core architectural strength, paradoxically contributes to user experience degradation as the action space expands beyond what can be efficiently browsed through hierarchical navigation alone.

5.1.2 Natural Language as an Interface for Service Requests

Natural language interfaces have demonstrated significant success in reducing user friction for complex systems across various domains, from web search to voice assistants to customer support chatbots. The application of natural language understanding to service request management offers several compelling advantages:

Alignment with User Mental Models: Users naturally formulate service

needs in descriptive language rather than navigating taxonomies. A statement like "The new hire needs access to the shared drives and email" directly expresses business intent without requiring translation to technical service categories. Natural language interfaces meet users where they are, eliminating the cognitive translation step from business need to IT taxonomy.

Reduced Time to Request Initiation: By accepting free-form descriptions and automatically mapping them to catalog actions, the system compresses the request initiation workflow from multi-step navigation to single-step description. This reduction in user effort directly improves adoption of self-service portals, reducing reliance on manual ticket creation through support channels.

Accessibility for Non-Technical Users: Business users without IT background can articulate needs in their own terminology, with the system handling the translation to technical service definitions. This democratization of self-service access reduces dependency on IT-literate intermediaries and distributes request creation capacity more broadly across the organization.

Contextual Understanding: Advanced natural language processing can extract contextual information from user descriptions, such as urgency indicators ("urgent," "as soon as possible"), scope qualifiers ("for the entire department"), or temporal constraints ("before the end of the week"), potentially enabling enhancements like automatic priority assignment or scheduling.

5.1.3 Explainability and Trust in Enterprise Environments

While natural language interfaces offer significant usability advantages, their application in enterprise IT service management introduces unique requirements around explainability and trust that distinguish this use case from consumer-facing recommendation systems [26]:

High-Stakes Decision Making: Service requests in enterprise environments often involve privileged operations (account creation, permission grants, resource provisioning) with security, compliance, and cost implications. Users must be able to validate that recommended actions align with their actual intent before submission, as erroneous requests may trigger unintended provisioning, create security vulnerabilities, or violate compliance policies.

Regulatory and Audit Requirements: Many enterprise IT operations are subject to regulatory frameworks (SOX, GDPR, HIPAA) requiring comprehensive audit trails and demonstrable controls [26]. Recommendation systems must provide transparent reasoning that enables post-hoc review of why specific actions were suggested, supporting compliance documentation and incident investigation [26].

User Confidence and Adoption: Enterprise users, particularly those in technical roles, exhibit skepticism toward "black box" AI systems that provide recommendations without explanation. Transparent reasoning builds user confidence in system suggestions and facilitates adoption by demonstrating that recommendations are grounded in relevant catalog information rather than arbitrary model outputs [26].

Error Correction and Learning: Explainable recommendations enable users to identify when the system has misunderstood their intent, providing opportunities for clarification and refinement [26]. If a recommendation system suggests "Delete User Account" when the user intended "Disable User Account," transparent reasoning ("This action permanently removes the user") enables immediate recognition of the mismatch and correction before request submission.

These explainability requirements motivate the architectural decision to implement a Retrieval Augmented Generation approach rather than relying on a Large Language Model alone. By explicitly retrieving relevant catalog entries and incorporating them into the generation context, the system ensures that recommendations are grounded in actual catalog content and can provide traceable reasoning linking user queries to specific action properties.

5.2 Large Language Models in Recommendation Systems

Large Language Models have emerged as powerful tools for natural language understanding and generation, demonstrating remarkable capabilities in tasks ranging from question answering to code generation. Their application to recommendation systems represents a significant departure from traditional collaborative filtering and content-based approaches, offering the potential for more nuanced understanding of user intent and more flexible matching between queries and catalog items.

5.2.1 Overview of Large Language Models

Large Language Models are neural networks trained on massive text corpora using self-supervised learning objectives, typically predicting the next token in a sequence given preceding context. Modern LLMs such as GPT-4 [30], Claude [5], and open-source alternatives like Llama [51] employ transformer architectures [56] with billions of parameters, enabling them to capture complex patterns in language use and world knowledge encoded in their training data.

The transformer architecture, introduced by Vaswani et al. [56], revolutionized natural language processing through its attention mechanism that enables

models to weigh the relevance of different parts of the input when generating outputs. This architecture allows LLMs to maintain context over long sequences and capture relationships between distant tokens, essential capabilities for understanding complex user queries and catalog descriptions.

Training these models typically follows a two-phase approach: pre-training on large-scale unlabeled text data using language modeling objectives, followed by fine-tuning on task-specific datasets or instruction-following examples [31]. Recent advances in instruction tuning and reinforcement learning from human feedback (RLHF) have significantly improved LLMs' ability to follow complex instructions and generate responses aligned with human preferences [31].

For the SRM recommendation system, the selected model is GPT-OSS-120B, a 120-billion parameter open-source model deployed on-premise within the company infrastructure. This deployment strategy addresses several enterprise requirements: data privacy concerns by ensuring that user queries and catalog information never leave the corporate network; compliance with regulatory frameworks prohibiting transmission of potentially sensitive service request descriptions to external API providers; cost control by eliminating per-request API fees; and customization potential through fine-tuning on company-specific data.

In addition, the system uses all-mpnet-base-v2 as the embedding model for semantic representation of service requests and catalog items. This model is also deployed on-premise, ensuring consistent privacy guarantees and tight integration with the internal infrastructure.

5.2.2 LLMs for Intent Understanding and Action Selection

The application of LLMs to the action recommendation task leverages several of their core capabilities that align naturally with the requirements of intent understanding and catalog matching:

Semantic Understanding Beyond Keywords: Unlike traditional keyword-based search systems that rely on lexical overlap between queries and documents, LLMs can understand the semantic intent behind user descriptions. A query like "The intern needs email access" is semantically equivalent to "Create a mailbox for a temporary employee" despite sharing no common keywords beyond "email." LLMs can recognize these semantic relationships through their learned representations of language meaning.

Contextual Disambiguation: Many terms in IT service management are polysemous or context-dependent. The word "access" might refer to VPN access, shared drive access, application access, or building access depending on context. LLMs can leverage surrounding context in the user query to

disambiguate intent, matching "remote access for traveling employees" to VPN-related actions while routing "access to the accounting system" to application permission actions.

Handling Underspecified Queries: Users often provide incomplete descriptions that nonetheless contain sufficient context for a human to infer intent. A query like "new user setup" might implicitly require account creation, mailbox provisioning, group memberships, and license assignment depending on organizational context. LLMs can leverage patterns learned during training to infer these implicit requirements and recommend comprehensive action sets.

Multi-Turn Refinement Potential: While the current implementation focuses on single-turn recommendations, LLMs enable future extensions to conversational interfaces where the system can ask clarifying questions ("Does the user need Office 365 access?" or "Is this a temporary or permanent account?") to narrow down recommendations. This capability emerges naturally from LLMs' instruction-following abilities without requiring explicit dialogue state tracking logic.

Reasoning and Explanation Generation: LLMs can generate natural language explanations for their recommendations, a critical requirement for enterprise deployment. By prompting the model to articulate why a particular action matches the user's query, the system produces human-readable justifications that build trust and enable error correction.

5.2.3 Limitations of Naked LLMs

Despite their impressive capabilities, deploying LLMs directly for action recommendation without retrieval augmentation introduces several significant limitations that compromise reliability and applicability in enterprise production environments:

Hallucination and Factual Inaccuracy: LLMs are prone to generating plausible-sounding but factually incorrect information, a phenomenon known as hallucination [21]. When asked to recommend actions from a catalog, an unaugmented LLM might invent action names, fabricate capability descriptions, or confidently recommend non-existent services. This unreliability is particularly problematic in enterprise contexts where users expect system outputs to reflect actual available services.

Knowledge Cutoff and Staleness: LLMs are frozen at the time of training, with no ability to incorporate new information without retraining or fine-tuning. For SRM, where the catalog evolves continuously through addition of new actions, retirement of obsolete services, and modification of existing workflows, an unaugmented LLM would quickly become outdated. A model trained in 2023 would have no knowledge of actions added to the catalog in

2024, making it unsuitable for production deployment without mechanisms to inject current catalog state.

Lack of Grounding in Catalog Data: LLMs generate recommendations based on patterns learned during training rather than explicit consultation of the current catalog. This means recommendations may reflect actions that were common in the training data but are not available in the specific SRM deployment, or conversely fail to surface recently added actions that match user needs perfectly. The absence of explicit grounding mechanisms prevents verification that recommendations correspond to actual catalog entries.

Inability to Handle Catalog-Specific Details: Enterprise service catalogs often contain highly specific actions with particular naming conventions, technical requirements, or organizational policies that would not be well-represented in general pre-training data. An LLM trained on broad internet text may struggle to distinguish between "Create User Account (Standard)" and "Create User Account (Privileged)" when both actions exist in the catalog with subtle but critical differences in provisioning logic.

Attribution and Auditability Challenges: Unaugmented LLM recommendations lack clear attribution to specific catalog entries or reasoning chains, making it difficult to audit why particular suggestions were made. For compliance and debugging purposes, organizations need to trace recommendations back to concrete catalog data and understand the matching logic, capabilities that "black box" LLM generation does not inherently provide.

Resource Consumption and Latency: Large-scale LLMs, particularly those with 120B parameters like GPT-OSS-120B, require substantial computational resources for inference. Generating recommendations through multiple forward passes or extensive prompt iterations introduces latency that degrades user experience in interactive applications. Retrieval mechanisms can reduce the inference burden by pre-filtering the action space before LLM processing.

These limitations motivate the adoption of Retrieval Augmented Generation, which addresses these concerns by explicitly retrieving relevant catalog entries and incorporating them into the model's generation context, ensuring that recommendations are grounded in current catalog state while maintaining the LLM's semantic understanding and explanation capabilities.

5.3 Retrieval Augmented Generation (RAG)

Retrieval Augmented Generation represents a paradigm shift in how Large Language Models access and utilize information, combining the semantic un-

derstanding capabilities of neural language models with the factual grounding and updatability of traditional information retrieval systems.

5.3.1 Core Concepts of Retrieval Augmented Generation

RAG, originally introduced by Lewis et al. [25], augments language model generation with a retrieval component that fetches relevant documents from an external knowledge base before generation. This architecture addresses the fundamental tension between parametric knowledge (information encoded in model weights during training) and non-parametric knowledge (information stored in external databases or document collections).

The key insight of RAG is that rather than requiring LLMs to memorize all facts within their parameters, we can provide models with access to external knowledge sources at inference time. This approach mirrors human information processing: rather than memorizing every fact, humans retrieve relevant information from external sources (books, databases, colleagues) when needed for decision-making or problem-solving.

The RAG architecture typically consists of three primary components:

1. **Document Store:** A repository containing the knowledge base documents. In the SRM context, representations of catalog actions including descriptions, extended descriptions, tags, categories, and other metadata. The document store must support efficient retrieval of relevant documents given a query.

2. **Retriever:** A component that accepts a query (the user's service request description) and returns the most relevant documents from the knowledge base. Modern retrievers typically employ dense vector representations computed by neural embedding models, enabling semantic similarity matching beyond keyword overlap.

3. **Generator:** A Large Language Model that receives both the original query and the retrieved documents, using this augmented context to generate the final output. In SRM's case, ranked action recommendations with explanatory reasoning.

This architecture provides several fundamental advantages over pure parametric models: knowledge can be updated by modifying the document store without retraining the LLM; the system can scale to arbitrarily large knowledge bases by improving retrieval efficiency rather than increasing model size; and generation is grounded in specific source documents, enabling attribution and verification.

5.3.2 How RAG Works

The RAG process for action recommendation in SRM follows a multi-stage pipeline that transforms user queries into grounded recommendations through retrieval and generation phases.

Document Embedding and Indexing

The first phase, typically executed offline during system initialization, involves converting catalog actions into dense vector representations and indexing them for efficient retrieval. Each action in the SRM catalog is transformed into a textual document containing its ID, description, extended descriptions (Italian and English), tags, action group, category, and status. These text representations are then processed through an embedding model that maps variable-length text to fixed-dimensional dense vectors.

Embedding models are neural networks trained to encode semantic similarity: texts with similar meanings should have embedding vectors that are close in the resulting vector space (typically measured by cosine similarity or Euclidean distance). Modern embedding models like sentence-transformers [38] are trained on large datasets of text pairs with similarity labels, learning to capture semantic relationships beyond superficial lexical overlap.

The generated embeddings are stored in a vector database (the system uses Chroma [10]), which indexes vectors using approximate nearest neighbor algorithms (such as HNSW [27]) that enable sub-linear time retrieval even as the catalog scales to thousands of actions. This indexing is critical for interactive performance: exact nearest neighbor search would require computing similarity between the query and every catalog action, while approximate algorithms achieve >95% recall with orders of magnitude better performance.

Query Embedding and Semantic Retrieval

When a user submits a service request description, the system first embeds the query using the same embedding model used for catalog indexing. This ensures that queries and catalog actions exist in the same semantic vector space, enabling meaningful similarity comparisons. The query embedding is a dense vector (typically 384-768 dimensions for modern sentence transformers) that encodes the semantic content of the user's description.

The vector database then performs approximate nearest neighbor search to identify the top-k most similar catalog actions to the query. This retrieval is semantic rather than lexical: the query "I need to recover a deleted email" can retrieve actions related to "Mailbox Recovery" or "Restore Email Items" even if the exact words differ, because the embedding model has learned that these phrases are semantically related through its training data.

The retrieval phase typically returns 5-10 candidate actions, balancing between providing sufficient context to the LLM and avoiding context window overflow. These candidates form the retrieval set that will be incorporated into the generation prompt.

Context Construction and Prompt Engineering

The retrieved candidate actions are formatted into a structured context document that will be provided to the LLM alongside the user's query. Effective prompt engineering is critical at this stage, as the prompt structure significantly influences the quality and format of LLM outputs [59].

The SRM system employs a carefully designed prompt template that includes: an instruction defining the task ("identify the TOP 3 most relevant actions"); the retrieved actions formatted with their IDs, descriptions, extended descriptions, groups, and tags; the user's original query; and a structured output format specification requiring JSON responses with action IDs, confidence scores, and reasoning.

This prompt structure implements several best practices from prompt engineering research: clear task instructions reduce ambiguity in what the model should generate [8]; providing concrete examples (few-shot learning) improves output quality, though not shown in the current implementation [8]; structured output formats enable reliable parsing of model responses [23]; and explicit reasoning requirements encourage chain-of-thought processing that improves answer quality [59].

LLM Generation with Retrieved Context

The LLM receives the augmented prompt containing both the user query and retrieved candidate actions, generating a response that ranks and explains the most relevant matches. The generation process leverages several mechanisms learned during training:

Attention over Retrieved Context: The transformer's attention mechanism [56] allows the model to selectively focus on relevant parts of the retrieved context when generating each token of the response. When generating the reasoning for why "Create User Account" matches the query "set up new employee," the model attends to the action's description and required fields.

Constrained Generation: The prompt explicitly constrains the output format to JSON with specific fields, leveraging the model's instruction-following capabilities developed through instruction tuning [31]. This produces structured outputs that can be reliably parsed by downstream systems.

Grounded Reasoning: By generating explanations based on information present in the retrieved context rather than parametric knowledge alone, the

model produces attributable reasoning that can be traced back to specific catalog properties. This grounding reduces hallucination and improves reliability.

The temperature parameter is set to 0 (deterministic sampling) to ensure consistent outputs across repeated queries, critical for production deployment where users expect reproducible recommendations.

Output Parsing and Validation

The LLM generates a JSON response containing ranked recommendations with action IDs, descriptions, confidence scores (0-1 scale), and brief reasoning. The system includes a cleaning step that removes common artifacts from LLM outputs: thinking tags (`<think>...</think>`) that some models include to show reasoning processes, markdown code block delimiters (`"‘json`), and extraneous text before or after the JSON object.

After cleaning, the response is parsed and validated to ensure it contains the required structure. Validation failures trigger error handling that returns the raw response for debugging, enabling rapid diagnosis of prompt engineering issues or unexpected model behaviors.

5.3.3 Problems Solved by RAG

The RAG architecture addresses the fundamental limitations of unaugmented LLMs identified in Section 2.3, providing several critical capabilities required for reliable production deployment:

Factual Grounding and Hallucination Mitigation: By explicitly providing current catalog actions in the generation context, RAG ensures that recommendations correspond to actual available services rather than invented actions. The model cannot hallucinate an action that doesn't exist in the retrieved set, as it is explicitly instructed to select from the provided candidates. This grounding dramatically reduces the hallucination problem that plagues pure LLM approaches.

Knowledge Freshness and Updatability: When new actions are added to the SRM catalog, they are immediately available for recommendation after re-indexing the vector database, a process requiring minutes rather than the days or weeks needed for model retraining. This updatability ensures the recommendation system remains synchronized with catalog evolution without requiring continuous model fine-tuning.

Transparent Attribution: RAG recommendations can be attributed to specific catalog entries present in the retrieval context. For each recommended action, the system can point to the exact catalog document that was retrieved and the properties that influenced the match. This transparency supports

auditability requirements and enables users to verify that recommendations are based on actual catalog content.

Reduced Computational Requirements: While RAG introduces a retrieval phase, it often reduces overall computational costs by narrowing the LLM's consideration space. Rather than processing the entire catalog (300+ actions) in the prompt, the system provides only the top-k most relevant candidates (typically 5-10), reducing prompt length and inference time. For extremely large catalogs, this filtering becomes essential for practical deployment.

Handling of Specialized Vocabulary: The retrieval phase, operating over catalog-specific text representations, naturally handles company-specific terminology, technical jargon, and naming conventions that may be poorly represented in the LLM's training data. Even if the LLM has never seen terms like "Privileged User Account" during pre-training, the retriever can match queries containing these terms to relevant catalog entries, with the LLM then leveraging its general language understanding to generate appropriate reasoning.

Separation of Concerns: RAG architectures cleanly separate the semantic matching problem (retrieval) from the ranking and explanation problem (generation). This modularity enables independent optimization of each component: improving retrieval quality through better embedding models or indexing strategies, and enhancing explanation quality through prompt engineering or model selection, without requiring coordinated changes across the system.

The combination of these capabilities makes RAG the architecture of choice for the SRM recommendation system, balancing the flexibility and semantic understanding of LLMs with the reliability and updatability requirements of enterprise production systems.

5.4 Design of the RAG-based Recommendation System in SRM

The SRM recommendation system implements a complete RAG pipeline exposed through a RESTful API built with FastAPI, enabling seamless integration with the existing MyElmec customer portal. The system is designed for deployment on company infrastructure alongside the on-premise GPT-OSS-120B language model, ensuring data sovereignty and eliminating external API dependencies.

5.4.1 High-Level Architecture

The recommendation system architecture, illustrated in Figure 5.1, consists of five primary components organized into initialization and inference phases.

Catalog Data Source

The system interfaces with the SRM backend API to retrieve the complete actions catalog for the target customer. The endpoint `/api/catalog/customers/{IDRS}/actions/details/` returns a JSON array containing all actions accessible to the customer, incorporating the three-level configuration hierarchy resolution. Each action entry includes comprehensive metadata: unique identifier, description, extended descriptions in Italian and English, tags, action group information, category details, and status.

This API-based catalog retrieval ensures that the recommendation system operates on the same effective catalog configuration visible to the customer through the standard UI, including any customer-specific or action-specific customizations. The use of an API endpoint rather than direct database access maintains separation of concerns and enables independent scaling of the recommendation service.

Document Preparation and Embedding

The `ActionMatcherRAG` class, implemented in Listing 20, transforms catalog actions into text documents suitable for embedding. For each action, the system constructs a comprehensive text representation concatenating all relevant fields, enabling the embedding model to capture semantic relationships across multiple properties.

The inclusion of both Italian and English extended descriptions accommodates the bilingual nature of the SRM deployment, where some customers operate primarily in Italian while others prefer English documentation. Tags and action group information provide categorical context that helps the embedding model understand relationships between actions within the organizational hierarchy.

Vector Store and Retrieval

The prepared documents are processed through the Chroma vector database [10], which handles both embedding generation and similarity search. The system employs the `sentence-transformers/all-MiniLM-L6-v2` model for embedding, a compact transformer model optimized for semantic similarity tasks [38]. This model maps text sequences to 384-dimensional dense vectors, achieving strong performance on semantic textual similarity benchmarks while maintaining computational efficiency suitable for real-time inference.

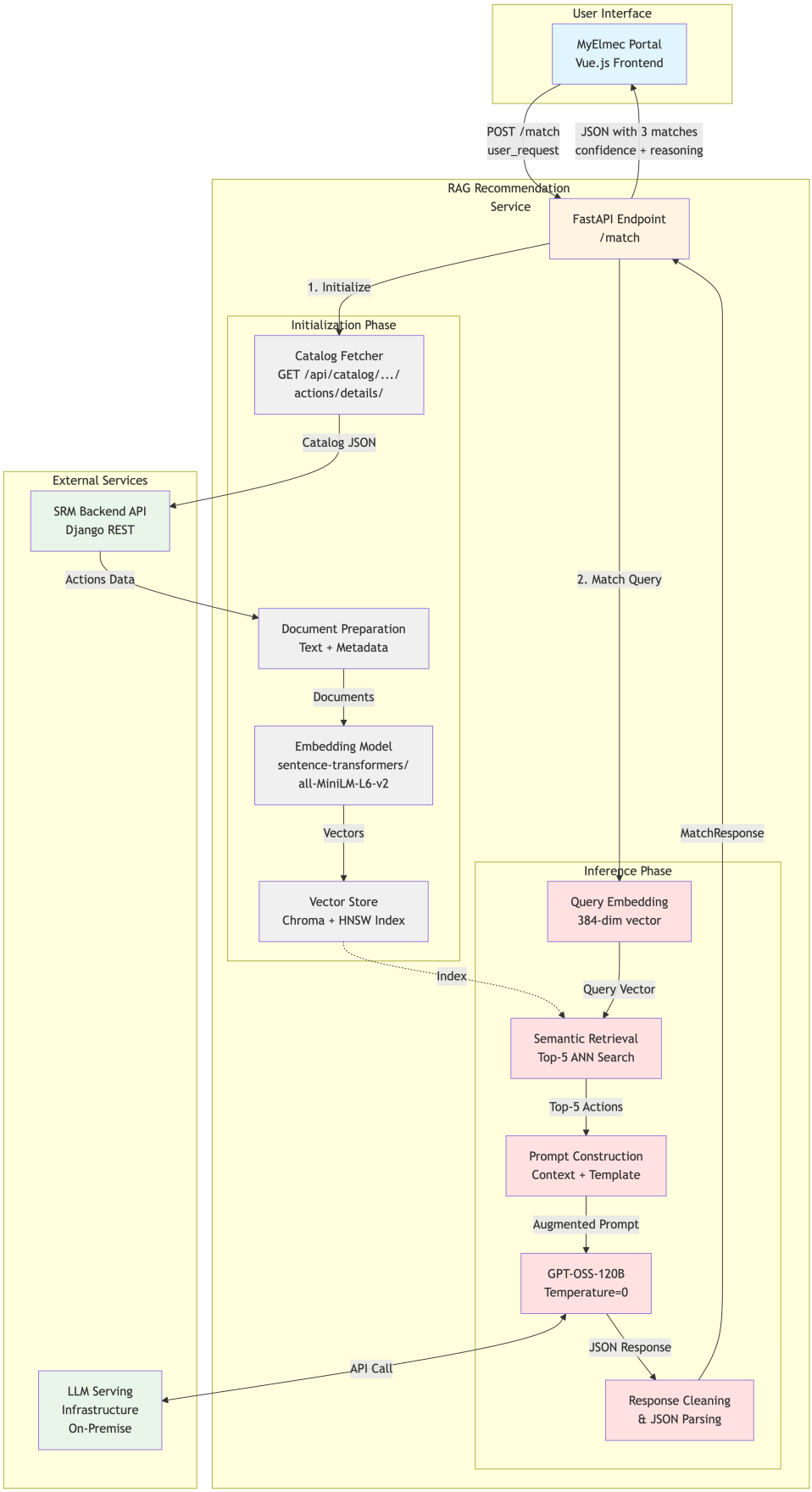

Figure 5.1: High-Level Architecture of the RAG-based Recommendation System

```
def prepare_documents(self, actions_data: List[Dict]) -> List[Document]:
    documents = []
    for action_item in actions_data:
        action = action_item.get("action", {})
        text_content = f"""
Action ID: {action.get('id')}
Description: {action.get('description', '')}
Extended Description (IT): {action.get('extended_description_it', '')}
Extended Description (EN): {action.get('extended_description_en', '')}
Tags: {action.get('tag', '')}
Action Group: {action.get('action_group', {}).get('description', '')}
# ...
        """
        metadata = {
            "action_id": action.get("id"),
            "description": action.get("description", ""),
            # ... additional metadata
        }
        documents.append(Document(page_content=text_content,
                                  metadata=metadata))
    return documents
```

Listing 20: Document Preparation for Catalog Actions

Document chunking is performed through LangChain's `RecursiveCharacterTextSplitter` with a chunk size of 1000 characters and 200-character overlap. This chunking strategy ensures that action descriptions exceeding the optimal embedding length are split appropriately while maintaining context continuity through overlap. For the SRM catalog, where most action descriptions are concise, chunking typically produces one or two chunks per action.

The retriever is configured to return the top-5 most semantically similar actions to a given query. This retrieval cardinality balances between providing sufficient candidates for the LLM to select from (enabling robust ranking) and avoiding context window overflow or introduction of irrelevant distractors that might confuse the ranking process.

Language Model Integration

The system integrates with GPT-OSS-120B through an OpenAI-compatible API interface exposed by the company's LLM serving infrastructure. The `ChatOpenAI` wrapper from LangChain provides a standardized interface that abstracts the specifics of the serving layer, enabling future model substitution without requiring changes to the RAG pipeline logic.

The temperature parameter is set to 0, enforcing deterministic sampling that produces consistent outputs for repeated queries. This determinism is critical for production deployment, where users expect reproducible recommendations and where inconsistent results would undermine trust in the system.

RAG Chain Composition

The RAG pipeline is implemented as a LangChain chain that composes retrieval, context formatting, prompting, and generation into a unified execution graph. Listing 21 demonstrates the chain construction using LangChain's declarative syntax.

```
def create_rag_chain(self) -> None:
    prompt = PromptTemplate(
        template=prompt_template,
        input_variables=["context", "question"]
    )

    def format_docs(docs):
        return "\n\n---\n\n".join([
            f"Action ID: {doc.metadata.get('action_id')}\n"
            f"Description: {doc.metadata.get('description')}\n"
            # ...
            for doc in docs
        ])

    self.rag_chain = (
        {"context": self.retriever | format_docs,
         "question": RunnablePassthrough()}
        | prompt
        | self.llm
        | StrOutputParser()
    )
```

Listing 21: RAG Chain Composition with LangChain

This declarative composition automatically handles data flow between components: the user query is embedded and used for retrieval, retrieved documents are formatted into context text, the prompt template is populated with both context and query, the completed prompt is sent to the LLM, and the raw text response is extracted. This abstraction eliminates boilerplate code and provides a clear representation of the data flow.

5.4.2 Embedding Strategy

The current implementation adopts an on-demand embedding strategy where the complete catalog is fetched, embedded, and indexed at the beginning of each recommendation request. This approach, while appearing inefficient, is justified by several pragmatic considerations specific to the SRM deployment context.

Catalog Scale and Embedding Performance

The SRM catalog for a typical customer contains 200-400 actions after configuration hierarchy resolution. Modern sentence transformer models like

all-MiniLM-L6-v2 can embed text sequences at rates exceeding 1000 sequences/second on modern GPUs or 100+ sequences/second on CPU-only deployments. Embedding the complete catalog thus requires 2-4 seconds on CPU or under 1 second on GPU, latencies that remain within acceptable bounds for interactive user interfaces.

The Chroma vector database construction and indexing add minimal overhead, as the approximate nearest neighbor index for collections of this scale can be built in sub-second timeframes. Total initialization time (catalog fetch + embedding + indexing) typically ranges from 5-10 seconds depending on API response time and hardware configuration, acceptable for an asynchronous recommendation workflow where users expect several seconds of processing time for "intelligent" suggestions.

Catalog Freshness and Consistency

The on-demand embedding strategy ensures that recommendations always reflect the current catalog state, including actions added, modified, or removed since the previous request. This freshness guarantee is particularly valuable during the SRP-SRM coexistence period where the catalog evolves through nightly synchronization jobs. A request at 14:00 will reflect catalog changes synchronized at 01:00 that morning, while a pre-built index updated every 3 hours might still contain stale action definitions.

The strategy also eliminates the complexity of cache invalidation logic that would be required for persistent vector stores. Determining when to refresh a cached index requires monitoring catalog modification timestamps, handling partial updates, and managing concurrent access during index rebuilds—engineering effort that outweighs the performance benefits for catalogs of SRM's scale.

Deployment Simplicity

On-demand embedding eliminates the need for persistent storage of vector indices, simplifying deployment to stateless container environments. The recommendation service can be deployed as a standard FastAPI application without requiring volume mounts, database migrations, or index initialization routines. This statelessness facilitates horizontal scaling, as multiple instances can be deployed without coordination or shared state management.

Future Optimization Path

The architecture is designed to accommodate future optimization through persistent indexing as the catalog grows. The modular separation between document preparation, embedding, and retrieval means that introducing a persistent vector store requires only localized changes to the `setup_vectorstore`

method. A caching strategy with periodic rebuilds (e.g., every 3 hours or on-demand via administrative API) could be implemented when catalog scale or request volume justifies the additional complexity.

The current on-demand approach thus represents a pragmatic trade-off that prioritizes simplicity, freshness, and deployment flexibility over raw performance optimization premature for the system's current usage patterns and scale.

5.5 Recommendation Flow

The recommendation flow orchestrates the complete pipeline from user query submission to structured recommendation delivery, integrating the RAG components described in previous sections into a cohesive end-to-end workflow.

5.5.1 User Query Processing

User interaction with the recommendation system occurs through a FastAPI endpoint that accepts natural language queries and returns structured JSON responses. Listing 22 shows the endpoint implementation.

```
@app.post("/match", response_model=MatchResponse)
async def match_action(request: MatchRequest) -> MatchResponse:
    logger.info(f"Processing: {request.user_request[:50]}...")

    matcher = ActionMatcherRAG(
        api_endpoint=API_ENDPOINT,
        lm_studio_url=LM_STUDIO_URL
    )

    matcher.initialize()  # Fetch catalog, embed, index
    result = matcher.match_action(request.user_request)

    if "error" in result:
        raise HTTPException(status_code=500,
                            detail=result.get("error"))

    return MatchResponse(**result)
```

Listing 22: FastAPI Endpoint for Action Matching

The endpoint leverages Pydantic models for request and response validation. The `MatchRequest` model enforces that user queries are non-empty strings with reasonable length constraints (1-1000 characters), while the `MatchResponse` model ensures that LLM outputs conform to the expected structure with exactly 3 matches, each containing an action ID, description, confidence score (0.0-1.0), and reasoning text.

This type-safe interface provides several benefits: automatic validation of incoming requests with clear error messages for malformed inputs; guaranteed structure of responses that frontend applications can parse reliably without defensive null checking; and OpenAPI/Swagger documentation auto-generation that facilitates frontend integration and testing.

The asynchronous handler design using `async def` enables the FastAPI server to handle multiple concurrent requests efficiently, important for production deployment where multiple users may request recommendations simultaneously. While the current implementation performs synchronous blocking calls within the handler (catalog fetch, embedding), the async interface provides a migration path to fully asynchronous execution through async HTTP clients and async LangChain components.

5.5.2 Semantic Retrieval of Candidate Actions

When a user query is submitted, the system first embeds it using the same sentence transformer model employed for catalog embedding, ensuring query and action representations exist in a shared semantic space. The query embedding is a 384-dimensional dense vector that encodes the semantic content of the user's request independent of surface-form lexical features.

The vector database performs approximate nearest neighbor search over the indexed action embeddings using the Hierarchical Navigable Small World (HNSW) algorithm [27]. HNSW constructs a multi-layer graph structure where nodes represent vectors and edges connect semantically similar vectors, enabling sub-linear time search through hierarchical traversal. For the SRM catalog scale (hundreds of actions), HNSW achieves >95% recall (finds 95% of true nearest neighbors) while examining only a small fraction of the total index.

The retrieval returns the top-5 most similar actions ranked by cosine similarity between query and action embeddings. Cosine similarity, defined as the dot product of normalized vectors, ranges from 0 (orthogonal, no similarity) to 1 (identical direction, perfect similarity). Actions with cosine similarity above typical thresholds (0.5-0.7) generally represent semantic matches worth presenting to the LLM for ranking.

5.5.3 Context-Augmented Prompt Construction

The retrieved candidate actions are formatted into a structured context document that will guide the LLM's ranking and explanation generation. Listing 23 shows the prompt template used for action matching.

The prompt template implements several prompt engineering best practices identified through iterative refinement: clear role definition ("AI assistant

```
You are an AI assistant specialized in matching user requests
to IT service actions.

Given the user's request and available actions from the catalog,
identify the TOP 3 most relevant actions.

Available Actions Context:
{context}

User Request: {question}

IMPORTANT: You MUST respond with a valid JSON object in this format:
{
    "matches": [
        {
            "action_id": <number>,
            "description": "<action description>",
            "confidence": <number between 0 and 1>,
            "reasoning": "<brief explanation>"
        }
    ],
    "user_request": "<original user request>"
}

Rules:
- Return exactly 3 matches, ordered by confidence (highest first)
- confidence must be between 0.0 and 1.0
- Keep reasoning brief (max 50 words)
- Respond ONLY with JSON, no additional text
```

Listing 23: Prompt Template for Action Recommendation

specialized in matching") establishes the model's persona and task context; structured output format with explicit schema reduces ambiguity in generation; concrete constraints (exactly 3 matches, confidence bounds, reasoning length) prevent common failure modes like generating too many/few results; and explicit instruction to respond only with JSON eliminates preambles and explanatory text that would break parsing.

The context section is populated with the formatted descriptions of the 5 retrieved candidate actions, each presented with its ID, description, extended description, group, and tags. This formatting provides the LLM with sufficient information to assess relevance while maintaining readability that supports the model's attention mechanisms in identifying key details.

5.5.4 LLM Output Interpretation

The LLM generates a JSON response following the prompt template structure, which undergoes cleaning and validation before being returned to the client. The cleaning process, shown in Listing 24, handles various formatting artifacts that LLMs commonly produce.

```
def clean_llm_response(self, response: str) -> str:
    response_clean = response.strip()

    # Remove thinking tags (e.g., from Qwen models)
    response_clean = re.sub(r"<think>.*?</think>", "",
                            response_clean, flags=re.DOTALL)

    # Remove markdown code blocks
    if response_clean.startswith("```json"):
        response_clean = response_clean[7:]
    elif response_clean.startswith("```"):
        response_clean = response_clean[3:]
    if response_clean.endswith("```"):
        response_clean = response_clean[:-3]

    # Extract JSON object
    json_start = response_clean.find("{")
    json_end = response_clean.rfind("}")
    if json_start != -1 and json_end != -1:
        response_clean = response_clean[json_start:json_end + 1]

    return response_clean
```

Listing 24: LLM Response Cleaning and JSON Extraction

This cleaning addresses several common LLM output patterns: thinking tags (`<think>...</think>`) that some models like Qwen use to expose reasoning processes; markdown code block delimiters (`"‘json`) that models trained to follow formatting conventions may include; and extraneous text before or after the JSON object that violates the "respond only with JSON" instruction.

After cleaning, the response is parsed as JSON and validated against the expected schema. The validation checks for presence of the `matches` array and `user_request` field, ensuring that the LLM followed the structured output format. Validation failures trigger error responses that include both the raw LLM output and the cleaned version, supporting rapid diagnosis of prompt engineering issues during development and production monitoring.

Successfully validated responses are wrapped in Pydantic `MatchResponse` objects that enforce type constraints (confidence between 0.0-1.0, exactly 3 matches, reasoning under 200 characters). This double validation—first at JSON structure level, then at Pydantic schema level—provides robust guarantees about response quality before delivery to frontend applications.

5.5.5 Current Limitations

Initialization Overhead

The on-demand embedding strategy, while ensuring catalog freshness and deployment simplicity, introduces 5-10 second latency for each recommendation request. This latency, while acceptable for an "intelligent assistant" feature where users expect some processing time, is noticeably longer than the sub-second response times users experience with traditional navigation. For high-frequency usage patterns where multiple users query simultaneously, this latency could degrade perceived system responsiveness.

The initialization overhead also prevents integration of recommendations into more interactive contexts like real-time suggestion during form field population or proactive recommendation based on user role and recent activity patterns. These use cases would require sub-second response times incompatible with the current architecture.

Single-Turn Interaction Model

The current system operates in a single-turn request-response pattern where the user submits a query and receives three recommendations without opportunity for refinement or clarification. Many real-world user intents are initially ambiguous or underspecified, requiring clarifying questions to narrow the action space. For example, a query "create new user" might benefit from follow-up questions like "Does the user need email access?" or "Is this a permanent or temporary account?" to distinguish between multiple user creation workflows.

The absence of conversational context management means each recommendation request is independent, preventing the system from learning from user corrections or refining suggestions based on rejected recommendations. If a user rejects all three suggestions and submits a new query, the system has no

memory of the previous attempt and cannot adjust its retrieval or ranking strategy accordingly.

Limited Multilingual Support

While the catalog embedding includes both Italian and English descriptions, the current prompt template and LLM configuration do not explicitly handle multilingual queries or ensure that reasoning is generated in the user's preferred language. A user submitting a query in Italian may receive reasoning in English, potentially reducing comprehension and trust. The sentence transformer model, while trained on multilingual data, may exhibit performance degradation for non-English queries compared to English inputs.

The system also lacks explicit language detection and routing logic that could select language-specific models or prompts optimized for the query language, potentially leaving performance on the table for Italian-speaking users who represent a significant portion of the SRM user base.

Absence of User Feedback Mechanisms

The current implementation lacks mechanisms for users to provide explicit feedback on recommendation quality—whether through binary relevance judgments (thumbs up/down), confidence corrections, or free-form comments. This absence prevents direct measurement of user satisfaction with recommendations and eliminates a valuable source of training data for future model improvements.

Without feedback collection, the system relies on implicit signals (whether users select recommended actions) to assess quality, but these signals are noisy: a user might select a suboptimal recommendation due to uncertainty rather than confidence, or abandon the interface due to UI confusion rather than recommendation failure. Explicit feedback would enable clearer signal extraction for system evaluation and improvement.

Static Ranking and Personalization

The recommendation system ranks actions solely based on semantic similarity between the query and catalog descriptions, without incorporating user-specific context like role, department, recent activity, or historical action selection patterns. Two users submitting identical queries receive identical recommendations regardless of whether one is an IT administrator who frequently provisions privileged accounts and the other is a department manager who typically requests standard user accounts.

This context-blindness represents a missed opportunity for personalization that could substantially improve recommendation relevance. Incorporating user context would enable the system to surface actions aligned with the user's

typical workflows and authority levels, reducing the cognitive load of evaluating recommendations and increasing selection rates for suggested actions.

5.5.6 Future Extensions

Several enhancement directions offer potential for substantial improvements to recommendation quality, user experience, and system capabilities.

Persistent Vector Store with Incremental Updates

Implementing a persistent vector store that is rebuilt periodically (e.g., every 3 hours) or triggered by catalog modification events would eliminate the initialization overhead from the critical path of recommendation requests. The system could maintain the vector index in memory across requests, reducing response latency from 5-10 seconds to under 1 second for the majority of cases.

Incremental update mechanisms could further optimize this approach by re-embedding only modified actions rather than rebuilding the entire index, enabling near-real-time catalog freshness with minimal computational overhead. Techniques like versioned indices with atomic swaps ensure that updates do not disrupt in-flight recommendation requests, maintaining service availability during index rebuilds.

Conversational Refinement Interface

Extending the system to support multi-turn conversations would enable clarifying questions and iterative refinement of recommendations. The LLM could identify ambiguous queries and generate targeted questions to narrow the action space, with the conversation history maintained in session state to inform subsequent retrieval and ranking.

This enhancement would require implementing dialogue state management, tracking which actions have been previously recommended and rejected, and designing prompts that leverage conversation context for progressive disambiguation. The LangChain framework provides conversation memory abstractions that could facilitate this extension with moderate engineering effort.

Fine-Tuning and Domain Adaptation

The current system uses GPT-OSS-120B as a general-purpose language model without domain-specific fine-tuning. Collecting user interaction data (queries, selected actions, explicit feedback) would enable supervised fine-tuning to better align the model with SRM-specific terminology, action relationships, and ranking preferences observed in real usage.

Fine-tuning could occur at multiple levels: training the embedding model on SRM-specific query-action pairs to improve semantic similarity estimation; instruction-tuning the LLM on examples of high-quality recommendations with reasoning; and potentially training a lightweight ranking model that reorders retrieval results based on user context features. These adaptations would improve recommendation quality without requiring architectural changes to the RAG pipeline.

Hybrid Retrieval Strategies

The current system relies solely on dense semantic retrieval through vector similarity. Hybrid approaches that combine dense retrieval with sparse retrieval (BM25, TF-IDF) have demonstrated superior performance on information retrieval benchmarks by capturing both semantic similarity and lexical overlap [29]. Implementing a hybrid retriever that merges results from dense and sparse indices could improve recall for queries containing specific technical terms or product names that are poorly captured by semantic embeddings alone.

Techniques like Reciprocal Rank Fusion [12] or learned reranking provide principled methods for combining retrieval signals from heterogeneous sources, potentially improving the quality of the candidate set provided to the LLM for final ranking.

Explainable AI and Reasoning Transparency

While the current system generates natural language reasoning for each recommendation, the reasoning is produced by the LLM without grounding in explicit interpretable features. Future enhancements could include feature-based explanations that identify specific catalog properties (tags, field requirements, group membership) that contributed to the match, visualized through highlighting or structured breakdowns.

Implementing attention visualization techniques that show which parts of the query and action description the model focused on during ranking could further enhance transparency, helping users understand the matching logic and building confidence in recommendations. These explainability enhancements would be particularly valuable for enterprise deployment where users expect transparency into automated decision-making systems.

Proactive Recommendation

The current system operates reactively, responding to explicit user queries. Proactive recommendation capabilities could surface relevant actions based on user context without requiring explicit queries: recommending common actions for new employees during onboarding, suggesting relevant actions

based on the user's department and role, or identifying actions frequently used by similar users (collaborative filtering).

These proactive capabilities would require integration with user profile data, activity tracking systems, and potentially training of user embedding models that map users to action preferences in a shared embedding space. The recommendation interface could present proactive suggestions as a "Recommended for You" section alongside the query input, providing value even before users articulate specific needs.

The combination of these enhancements would evolve the system from a query-driven action lookup tool into a comprehensive intelligent assistant for service request management, combining semantic understanding, personalization, and proactive guidance to substantially reduce friction in the service request creation workflow.

Conclusions

This thesis presented the complete redesign and implementation of Elmec Informatica's Service Request management system, successfully transforming a legacy distributed monolith into a modern, scalable microservices architecture. The project addressed critical limitations in the legacy SRP system while maintaining business continuity through a carefully orchestrated migration strategy.

The architectural redesign resolved fundamental issues that had accumulated over years of organic system growth. The monolithic frontend component, previously exceeding 11,000 lines of hard-coded business logic, was reduced to a lean 1,000-line interface through centralization of configuration logic in the backend. The three-level configuration hierarchy (Fields → Actions → Customers) introduced unprecedented flexibility in catalog management, enabling customer-specific customizations without code duplication or catalog fragmentation.

The elimination of the SRP Agent dependency represented a major architectural breakthrough. Integration with Provision Prime's Ansible-based provisioning system removed security risks associated with domain-admin credentials on customer infrastructure, eliminated polling-based communication inefficiencies, and enabled centralized logging through S3-compatible object storage. This transition from agent-based to agentless execution fundamentally improved system reliability and operational transparency.

The migration from unversioned Stored Procedures and SSIS jobs to version-controlled Python code within a comprehensive CI/CD pipeline transformed development practices. The implementation of automated quality gates, security scanning, and environment-specific deployment workflows ensures code quality while enabling rapid iteration. The adoption of PostgreSQL with properly enforced referential integrity constraints eliminated the data inconsistencies that plagued the legacy SQL Server database.

The RAG-enhanced recommendation system represents a significant innovation in enterprise service request management. By combining Large Language Models with semantic retrieval over the catalog, the system enables natural language-based action discovery that dramatically reduces cognitive load for end users. The architecture's emphasis on explainability through transparent reasoning addresses the trust requirements critical for enterprise adoption, distinguishing this implementation from consumer-facing recommendation systems.

The phased coexistence strategy successfully balanced innovation with operational stability, enabling incremental validation through internal pilots, customer pilots, and progressive expansion. This risk-managed approach avoided the disruptions common in "big bang" migrations while building organizational confidence in the new architecture.

Future enhancements to the recommendation system, including persistent vector stores, conversational refinement, and personalized ranking, offer clear paths for continued improvement. The modular architecture facilitates these extensions without requiring fundamental redesign, demonstrating the system's capacity for evolution alongside emerging technologies and changing business requirements.

This work demonstrates that legacy enterprise systems, despite accumulated technical debt and architectural constraints, can be successfully modernized through careful analysis, principled design, and pragmatic migration strategies. The ServiceRequestManager stands as evidence that substantial architectural improvements are achievable while maintaining the business continuity essential in production environments serving large enterprise clients.

Bibliography

[1] *Active Directory Organizational Unit (OU)*. URL: https://blog.netwrix.com/2024/02/19/active-directory-organizational-unit-ou/ (cit. on p. 21).

[2] Mashel Albarak and Rami Bahsoon. *Prioritizing Technical Debt in Database Normalization Using Portfolio Theory and Data Quality Metrics*. 2018. arXiv: 1801.06989 [cs.SE]. URL: https://arxiv.org/abs/1801.06989 (cit. on p. 14).

[3] Charles Anderson. "Docker [Software engineering]". In: *IEEE Software* 32.3 (2015), pp. 102–c3. DOI: 10.1109/MS.2015.62 (cit. on pp. 26, 27).

[4] *Ansible Documentation*. URL: https://docs.ansible.com/ansible/latest/dev_guide/overview_architecture.html (cit. on pp. 32, 41).

[5] *Anthropic - Introducing the Next Generation of Claude*. URL: https://www.anthropic.com/news/claude-3-family (cit. on p. 60).

[6] *Argo CD - Declarative GitOps CD for Kubernetes*. URL: https://argo-cd.readthedocs.io/en/stable/ (cit. on p. 38).

[7] Michael R. Blaha. "Referential Integrity Is Important For Databases". In: 2005. URL: https://api.semanticscholar.org/CorpusID:6831872 (cit. on p. 14).

[8] Tom B. Brown et al. *Language Models are Few-Shot Learners*. 2020. arXiv: 2005.14165 [cs.CL]. URL: https://arxiv.org/abs/2005.14165 (cit. on p. 66).

[9] *Celery - Distributed Task Queue*. URL: https://docs.celeryq.dev/en/stable/getting-started/introduction.html (cit. on p. 41).

[10] *Chroma - Documentation*. URL: https://docs.trychroma.com/docs/overview/introduction (cit. on pp. 65, 69).

[11] *Connection Pooling for Postgres using PG Bouncer*. URL: https://medium.com/@pablo.lopez.santori/connection-pooling-for-postgres-using-pg-bouncer-175bc1607db2 (cit. on p. 41).

[12] Buttcher Cormack Clarke. "Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods". In: (2009) (cit. on p. 81).

[13] *Cryptsetup and LUKS - open-source disk encryption*. URL: https://gitlab.com/cryptsetup/cryptsetup (cit. on p. 10).

[14] Lorenzo De Lauretis. "From Monolithic Architecture to Microservices Architecture". In: *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. 2019, pp. 93–96. DOI: 10.1109/ISSREW.2019.00050 (cit. on p. 22).

[15] *Django - The web framework for perfectionists with deadlines*. URL: https://www.djangoproject.com/ (cit. on p. 40).

[16] *Django REST framework*. URL: https://www.django-rest-framework.org/ (cit. on p. 40).
[17] *DMI Infrastructure*. Internal repository, not publicly accessible. URL: https://git.elmec.com/publicautomation/dmi-infrastructure.git (cit. on pp. 7, 10).
[18] Roy T. Fielding. *Architectural Styles and the Design of Network-Based Software Architectures*. University of California, Irvine, 2000 (cit. on p. 24).
[19] Paul J. Deitel Harvey M. Deitel. *C++. Fondamenti di programmazione*. Apogeo, 2005 (cit. on p. 14).
[20] *HDA - Help Desk Advanced*. URL: https://pat.eu/en/products/helpdeskadvanced/ (cit. on p. 25).
[21] Ziwei Ji et al. "Survey of Hallucination in Natural Language Generation". In: *ACM Computing Surveys* 55.12 (Mar. 2023), pp. 1–38. ISSN: 1557-7341. DOI: 10.1145/3571730. URL: http://dx.doi.org/10.1145/3571730 (cit. on p. 62).
[22] *K3s - Lightweight Kubernetes*. URL: https://docs.k3s.io/ (cit. on p. 10).
[23] Takeshi Kojima et al. *Large Language Models are Zero-Shot Reasoners*. 2023. arXiv: 2205.11916 [cs.CL]. URL: https://arxiv.org/abs/2205.11916 (cit. on p. 66).
[24] *Kubernetes: Production-Grade Container Orchestration*. URL: https://kubernetes.io/docs/concepts/overview/ (cit. on pp. 26, 27).
[25] Patrick Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 2021. arXiv: 2005.11401 [cs.CL]. URL: https://arxiv.org/abs/2005.11401 (cit. on p. 64).
[26] Indraneel Madabhushini. "Explainable AI (XAI) in Business Intelligence: Enhancing Trust and Transparency in Enterprise Analytics". In: *Emerging Frontiers Library for The American Journal of Engineering and Technology* 7.8 (2025), pp. 9–20 (cit. on pp. 59, 60).
[27] Yu. A. Malkov and D. A. Yashunin. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. 2018. arXiv: 1603.09320 [cs.DS]. URL: https://arxiv.org/abs/1603.09320 (cit. on pp. 65, 75).
[28] *Memcached - A distributed memory object caching system*. URL: https://docs.memcached.org/ (cit. on p. 41).
[29] Anthony Mudet and Souhail Bakkali. *Hybrid Retrieval-Augmented Generation for Robust Multilingual Document Question Answering*. 2025. arXiv: 2512.12694 [cs.DL]. URL: https://arxiv.org/abs/2512.12694 (cit. on p. 81).
[30] OpenAI et al. *GPT-4 Technical Report*. 2024. arXiv: 2303.08774 [cs.CL]. URL: https://arxiv.org/abs/2303.08774 (cit. on p. 60).
[31] Long Ouyang et al. *Training language models to follow instructions with human feedback*. 2022. arXiv: 2203.02155 [cs.CL]. URL: https://arxiv.org/abs/2203.02155 (cit. on pp. 61, 66).

[32] Severi Peltonen, Luca Mezzalira, and Davide Taibi. *Motivations, Benefits, and Issues for Adopting Micro-Frontends: A Multivocal Literature Review*. 2021. arXiv: 2007.00293 [cs.SE]. URL: https://arxiv.org/abs/2007.00293 (cit. on pp. 11, 24).

[33] *PEP 8 – Style Guide for Python Code*. URL: https://peps.python.org/pep-0008/ (cit. on p. 38).

[34] *Pinia - The intuitive store for Vue.js*. URL: https://pinia.vuejs.org/introduction.html (cit. on p. 47).

[35] *Pod Quality of Service Classes*. URL: https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/ (cit. on p. 39).

[36] *Provision Prime*. Internal repository, not publicly accessible. URL: https://git.elmec.com/innovation/development/provisionprime.git (cit. on pp. 21, 22, 25).

[37] *Refactoring Guru - Strategy Pattern*. URL: https://refactoring.guru/design-patterns/strategy (cit. on p. 32).

[38] Nils Reimers and Iryna Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. 2019. arXiv: 1908.10084 [cs.CL]. URL: https://arxiv.org/abs/1908.10084 (cit. on pp. 65, 69).

[39] David K. Rensin. *Kubernetes - Scheduling the Future at Cloud Scale*. 2015, All. URL: http://www.oreilly.com/webops-perf/free/kubernetes.csp (cit. on pp. 27, 28).

[40] Elmec Informatica S.p.A. *Azioni Gestione SRP 4.0*. Internal document, not publicly available., 2016 (cit. on pp. 3–5, 25).

[41] Elmec Informatica S.p.A. *Requirements definition meeting for the new system*. Internal meeting. Held with the stakeholders and developers of the old system to gather requirements for the new system. Apr. 2025 (cit. on pp. 7, 14, 17–22).

[42] Elmec Informatica S.p.A. *Service Request Portal*. Internal document, not publicly available., 2016 (cit. on pp. 7–9, 20).

[43] *Semgrep App Security Platform | AI-Assisted SAST, SCA and Secrets Detection*. URL: https://semgrep.dev/ (cit. on p. 38).

[44] *SQL Server Integration Services*. URL: https://learn.microsoft.com/en-us/sql/integration-services/sql-server-integration-services (cit. on p. 13).

[45] *SRP Internal Worker Windows Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/srpinternalworker_windowsservice.git (cit. on p. 9).

[46] *SRP Web Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/SRP_WebService.git (cit. on p. 8).

[47] *SRP Web Service REST*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/srp_webservicerest.git (cit. on pp. 8, 18, 20, 22).

[48] *SRP Windows Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/SRP_WindowsService.git (cit. on pp. 6, 9, 14, 21, 22).

[49] Michael Stonebraker, Lawrence Rowe, and Michael Hirohama. "The Implementation Of Postgres". In: *Knowledge and Data Engineering, IEEE Transactions on* 2 (Apr. 1990), pp. 125–142. DOI: 10.1109/69.50912 (cit. on p. 33).

[50] *Stored Procedures (Database Engine)*. URL: https://learn.microsoft.com/en-us/sql/relational-databases/stored-procedures/stored-procedures-database-engine (cit. on p. 13).

[51] Hugo Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023. arXiv: 2302.13971 [cs.CL]. URL: https://arxiv.org/abs/2302.13971 (cit. on p. 60).

[52] Jan Tretmans and Louis Verhaard. "A Queue Model Relating Synchronous and Asynchronous Communication". In: *Protocol Specification, Testing and Verification, XII*. Ed. by R.J. LINN and M.Ü. UYAR. IFIP Transactions C: Communication Systems. Amsterdam: Elsevier, 1992, pp. 131–145. ISBN: 978-0-444-89874-6. DOI: https://doi.org/10.1016/B978-0-444-89874-6.50015-5. URL: https://www.sciencedirect.com/science/article/pii/B9780444898746500155 (cit. on p. 25).

[53] *Trivy - The all-in-one open source security scanner*. URL: https://trivy.dev/ (cit. on p. 38).

[54] *TypeScript: JavaScript with Syntax for Types*. URL: https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html (cit. on p. 28).

[55] Brecht Van de Vyvere, Pieter Colpaert, and Ruben Verborgh. "Comparing a Polling and Push-Based Approach for Live Open Data Interfaces". In: *Web Engineering*. Ed. by Maria Bielikova, Tommi Mikkonen, and Cesare Pautasso. Cham: Springer International Publishing, 2020, pp. 87–101. ISBN: 978-3-030-50578-3 (cit. on p. 20).

[56] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762 (cit. on pp. 60, 66).

[57] *Vault - Manage Secrets and Protect Sensitive Data*. URL: https://developer.hashicorp.com/vault (cit. on p. 39).

[58] *Vue.js v3 Documentation*. URL: https://vuejs.org/guide/introduction (cit. on pp. 28, 29, 47).

[59] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2023. arXiv: 2201.11903 [cs.CL]. URL: https://arxiv.org/abs/2201.11903 (cit. on p. 66).

[60] *What is a Container?* URL: https://www.docker.com/resources/what-container/ (cit. on p. 27).

[61] *What is referential integrity and what does it look like in practice?* URL: https://www.y42.com/blog/referential-integrity (cit. on p. 14).

[62] *Xenon - Monitoring tool based on radon.* URL: `https://github.com/rubik/xenon` (cit. on p. 38).